FC7xxx FEE User Manual

Rev.A1

Revision History

Revision	Date	Changes
A0	2025/01/15	Initial release for MCAL V0.1.0
A1	2025/04/25	Update software implementation and user manual need update too

Table of Contents

Chapter 1 FEE Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of Flash EEPROM Emulation(FEE) specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_Flash EEPROM Emulation.*

1.2 Design Summary

EEPROM (electrically erasable programmable read only memory), which can be byte or word programmed and erased, is often used in automotive electronic control units. For many MCUs which do not contain on-chip EEPROM. FEE module is often used to replace this.

The FEE module is a pure software of memory management algorithm provides services for reading, writing, erasing and invalidating emulated EEPROM blocks apart from other basic features specified by the software specification in flash. All these services depend on the implements of two logic sectors cyclic swap, during the FEE module configuration each logic sector will be confogured as one or more physical flash sector.

The memory related operations are performed asynchronously when using the FEE module. Their respective APIs store actual parameters into internal data structures and immediately return. The job is performed by means of state machines, which are driven by repetitive calls to the so called main functions of the FEE (Fee_MainFunction). These executive functions accomplish their tasks in predefined chunks of data only, and shall be called by the application repeatedly (e.g. periodically in a dedicated task).

Note: For normally use of FEE module, the FLS driver must be correct configured in advance.

1.3 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of FLAGCHIP Semiconductors:

- FC7300 series including FC7300F8MDQ, FC7300F8MDT, FC7300F4MDD and FC7300F8MDS with different packages.
- FC7240F2MDS with different packages.

1.4 Hardware Resources

The FEE module is hardware independent module and depends on the underlying FLS module and its configuration only.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 SWS_Fee_00022	
Description	If the current module status is MEMIF_IDLE or if the current module status is MEMIF_BUSY INTERNAL, the function Fee_Read shall accept the read request, copy the given / computed parameters to module internal variables, initiate a read job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK.
Rejection Reason	MEMIF_BUSY INTERNAL is used only for the Fee_Init job. In case the status is MEMIF_BUSY INTERNAL Fee will reject the job.

Rejected Requirement 2 SWS_Fee_00025	
Description	If the current module status is MEMIF_IDLE or if the current module status is MEMIF_BUSY INTERNAL, the function Fee_Write shall accept the write request, copy the given / computed parameters to module internal variables, initiate a write job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK.
Rejection Reason	MEMIF_BUSY INTERNAL is used only for the Fee_Init job. In case the status is MEMIF_BUSY INTERNAL Fee will reject the job.

Rejected Requirement 4 SWS_Fee_00102	
Description	The configuration of the FEE module shall define the expected number of erase/write cycles for each logical block in the configuration parameter FeeNumberOfWriteCycles.
Rejection Reason	The FeeNumberOfWriteCycles could not be set but depend on write times of the block in the same logic sector.

Rejected Requirement 5 SWS_Fee_00104	
Description	API function Description Det_ReportError Service to report development errors. Fls_BlankCheck. The function Fls_BlankCheck shall verify, whether a given memory area has been erased but not (yet) programmed. The function shall limit the maximum number of checked flash cells per main function cycle to the configured value FlsMaxReadNormalMode or FlsMaxReadFastMode respectively.
Rejection Reason	Do not support this interface.

Rejected Requirement 6 SWS_Fee_00187	
Description	If the function Fls_BlankCheck is configured (in the flash driver), the function Fee_Read shall call the function Fls_BlankCheck to determine in advance whether a given memory area can be read without encountering e.g. ECC errors due to trying to read erased but not programmed flash cells.
Rejection Reason	Do not support this interface.

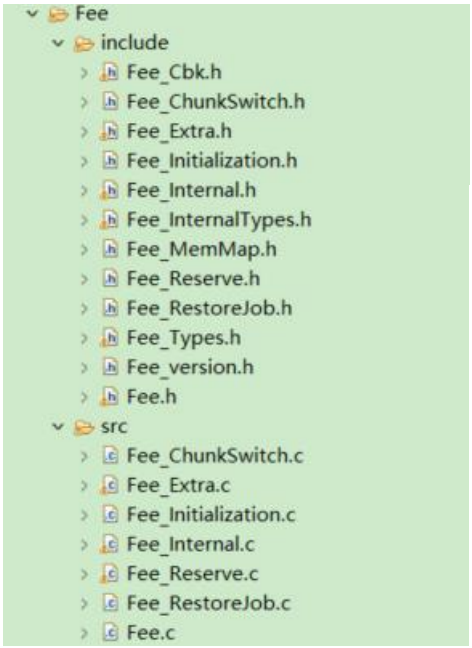
Rejected Requirement 7 SWS_Fee_00188		
Description	**Name**	Fee_ConfigType

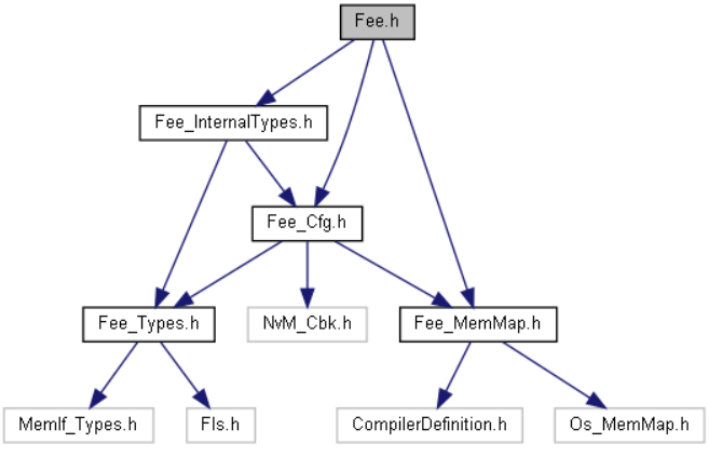
	Type	Structure
	Range	implementation
	Description	Configuration data structure of the Fee module.
Rejection Reason	Fee_ConfigType is always NULL_PTR	

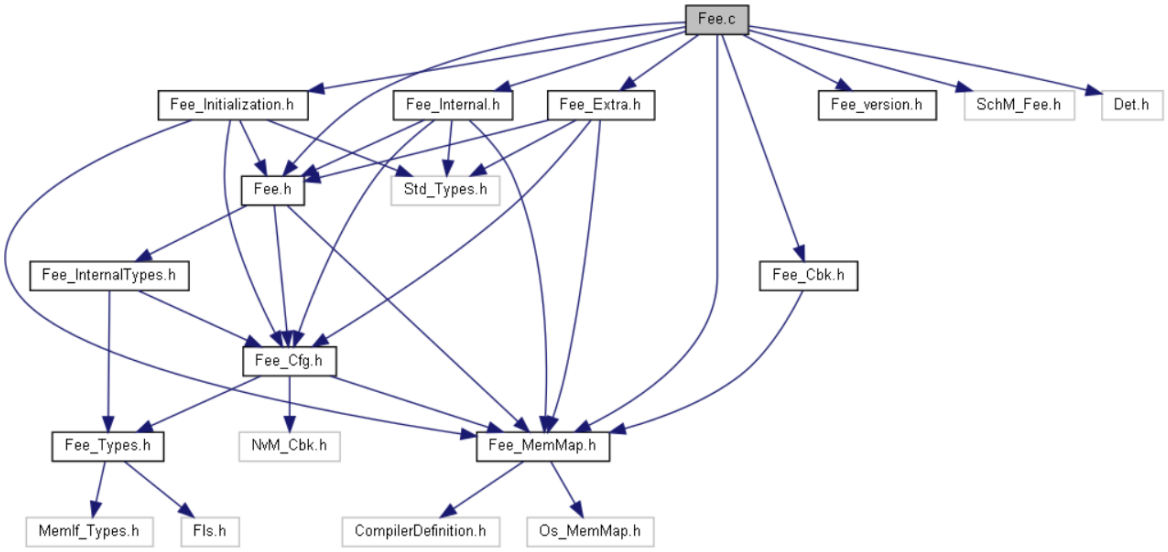
Rejected Requirement 8 SWS_Fee_00192	
Description	The function Fee_InvalidateBlock shall check if the module state is MEMIF_IDLE or MEMIF_BUSY_INTERNAL. If this is the case the module shall accept the invalidation request and shall return E_OK to the caller. The block invalidation shall be executed asynchronously in the module's main function as soon as the module has finished the internal management operation.
Rejection Reason	MEMIF_BUSY INTERNAL is used only for the Fee_Init job.In case the status is MEMIF_BUSY INTERNAL Fee will reject the job.

Rejected Requirement 9 ECUC_Fee_00110		
Description	**Name**	FeeNumberOfWriteCycles
	Parent Container	FeeBlockConfiguration
	Description	Number of write cycles required for this block.
	Multiplicity	1
	Type	EcucIntegerParamDef
	Range	0 ... 4294967295
	Default value	--
	Post-Build Variant Value	false
Rejection Reason	Do not support FeeNumberOfWriteCycles	

2.2 File Structure







[SWDESG_FEE_088] The file include structure shall follow above.

2.3 Macros

2.3.1 Macros in Fee.h

- #define **FEE_INSTANCE_ID** ((uint8)0U)
  Fee instance ID.

- #define **FEE_INIT_ID** ((uint8)0x00U)
  AUTOSAR API service ID - Fee init.

- #define **FEE_SETMODE_ID** ((uint8)0x01U)
  AUTOSAR API service ID - Fee set mode.

- #define **FEE_READ_ID** ((uint8)0x02U)
  AUTOSAR API service ID - Fee read.

- #define **FEE_WRITE_ID** ((uint8)0x03U)
  AUTOSAR API service ID - Fee write.

- #define **FEE_CANCEL_ID** ((uint8)0x04U)
- AUTOSAR API service ID - Fee cancel.

- #define **FEE_GETSTATUS_ID** ((uint8)0x05U)
  AUTOSAR API service ID - Fee get status.

- #define **FEE_GETJOBRESULT_ID** ((uint8)0x06U)
  AUTOSAR API service ID - Fee get job result.

- #define **FEE_INVALIDATEBLOCK_ID** ((uint8)0x07U)
  AUTOSAR API service ID - Fee invalidate block.

- #define **FEE_GETVERSIONINFO_ID** ((uint8)0x08U)
  AUTOSAR API service ID - Fee get version.

- #define **FEE_ERASEIMMEDIATEBLOCK_ID** ((uint8)0x09U)
  AUTOSAR API service ID - Fee erase immediate block.

- #**define FEE_JOBENDNOTIFICATION_ID** ((uint8)0x10U)
  AUTOSAR API service ID - Fee job end notification.

- #define **FEE_JOBERRORNOTIFICATION_ID** ((uint8)0x11U)
  AUTOSAR API service ID - Fee job error notification.

- #define **FEE_MAINFUNCTION_ID** ((uint8)0x12U)
  AUTOSAR API service ID - Fee mainfunction.

- #define **FEE_FORCECHUNKSWITCHONNEXTWRITE_ID** ((uint8)0x13)
  Non-AUTOSAR API service ID - Fee_ForceSwapOnNextWrite.

- #define **FEE_E_UNINIT** ((uint8)0x01U)
  API service called when module was not initialized.

- #define **FEE_E_INVALID_BLOCK_NO** ((uint8)0x02U)
  API called with invalid block number.

- #define **FEE_E_INVALID_BLOCK_OFS** ((uint8)0x03U)
  API called with invalid block offset.

- #define **FEE_E_PARAM_POINTER** ((uint8)0x04U)
  API called with invalid data pointer.

- #define **FEE_E_INVALID_BLOCK_LEN** ((uint8)0x05U)
  API called with invalid length information.

- #define **FEE_E_BUSY** ((uint8)0x06U)
  API called while module is busy processing a user request.

- #define **FEE_E_BUSY_INTERNAL** ((uint8)0x07U)
  API called while module is busy doing internal management operations.

- #define **FEE_E_INVALID_CANCEL** ((uint8)0x08U)
  API called while module is not busy because there is no job to cancel.

- #define **FEE_E_INIT_FAILED** ((uint8)0x09U)
  API Fee_init failed.

- #define **FEE_E_CANCEL_API** ((uint8)0x0AU)
  API called when underlying driver has cancel api disabled.

2.3.2 Macros in Fee_Extra.h

- #define **FEE_CHUNK_ID_INDEX** 0U
  Index of chunk ID information.

- #define **FEE_CHUNK_ADDRESS_INDEX** 4U
  Index of chunk start address information.

- #define **FEE_CHUNK_SIZE_INDEX** 8U
  Index of chunk size information.

- #define **FEE_CHUNK_CHECKSUM_INDEX** 12U
  Index of chunk checksum information.

- #define **FEE_CHUNK_HEADER_PART1_SIZE** 16U
  Size of chunk header part 1. More...

- #define **FEE_BLOCK_ID_INDEX** 0U
  Index of block ID information.

- #define **FEE_BLOCK_SIZE_INDEX** 2U
  Index of block size information.

- #define **FEE_BLOCK_ADDRESS_INDEX** 4U
  Index of the start address of the block data.

- #define **FEE_BLOCK_CHECKSUM_INDEX** 8U
  Index of block header checksum information.

- #define **FEE_BLOCK_ASSIGNMENT_INDEX** 12U
  Index of block assignment information.

- #define **FEE_BLOCK_IMMED_MASK** 0x80000000U
  Mask of immediate info in block header checksum

- #define **FEE_BLOCK_IMMED_USED_MASK** 0x7FFFFFFFU
  Mask of immediate info in block header checksum

- #define **FEE_READ_BYTE**(Buffer, Index) ((uint8)(Buffer)[(Index)])
  Macro to read the byte from the uint8 buffer.

  #define **FEE_READ_HALFWORD**(Buffer, Index)
  ((uint16)((uint16)((Buffer)[(Index)+1U])<<8U)|(uint16)((Buffer)[(Index)]))
  Macro to read half word from the uint8 buffer.

- #define **FEE_READ_WORD**(Buffer, Index)
  Macro to read word from the uint8 buffer. More...

- #define **FEE_WRITE_BYTE**(Buffer, Index, Byte) (Buffer)[(Index)] = (uint8)(Byte);
  Macro to write the byte to the uint8 buffer. More...

- #define **FEE_WRITE_HALFWORD**(Buffer, Index, Word)

Macro to write half word to the uint8 buffer. More...

- #define **FEE_WRITE_WORD**(Buffer, Index, Word)
  Macro to write word to the uint8 buffer.

2.3.3 Macros in Fee_InternalTypes.h

- #define **FEE_NXT_JOB_READ** (Fee_JobType)0
  Read Fee block.

- #define **FEE_NXT_JOB_WRITE** (Fee_JobType)1
  Write Fee block to flash.

- #define **FEE_NXT_JOB_WRITE_DATA** (Fee_JobType)2
  Write Fee block data to flash.

- #define **FEE_NXT_JOB_WRITE_UNALIGNED_DATA** (Fee_JobType)3
  Write unaligned rest of Fee block data to flash.

- #define **FEE_NXT_JOB_WRITE_BLOCK_HDR_VLD** (Fee_JobType)4
  Validate Fee block by writing validation flag to flash.

- #define **FEE_NXT_JOB_WRITE_DONE** (Fee_JobType)5
  Finalize validation of Fee block.

- #define **FEE_NXT_JOB_INVAL_BLOCK** (Fee_JobType)6
  Invalidate Fee block by writing the invalidation flag to flash.

- #define **FEE_NXT_JOB_INVAL_BLOCK_DONE** (Fee_JobType)7
  Finalize invalidation of Fee block.

- #define **FEE_NXT_JOB_ERASE_IMMEDIATE** (Fee_JobType)8
  Erase (pre-allocate) immediate Fee block.

- #define **FEE_NXT_JOB_INIT_SCAN** (Fee_JobType)9
  Initialize the chunk scan job.

- #define **FEE_NXT_JOB_INIT_SCAN_CHUNK** (Fee_JobType)10
  Scan active chunk of current chunk group.

- #define **FEE_NXT_JOB_INIT_SCAN_CHUNK_HDR_RESOLVE** (Fee_JobType)11
  Parse Fee chunk header.

- #define **FEE_NXT_JOB_INIT_SCAN_CHUNK_HDR_PROGRAM** (Fee_JobType)12
  Format first Fee chunk.

- #define **FEE_NXT_JOB_INIT_SCAN_CHUNK_HDR_PROGRAM_DONE** (Fee_JobType)13
  Finalize format of first Fee chunk.

- #define **FEE_NXT_JOB_INIT_SCAN_BLOCK_HDR_RESOLVE** (Fee_JobType)14

Parse Fee block header.

- #define **FEE_NXT_JOB_CS_CHUNK_HDR_PROGRAM** (Fee_JobType)15
  Format current Fee chunk in current Fee chunk group.

- #define **FEE_NXT_JOB_CS_COPY_BLOCK** (Fee_JobType)16
  Copy next block from source to target chunk.

- #define **FEE_NXT_JOB_CS_COPY_DATA_READ** (Fee_JobType)17
  Read data from source chunk to internal Fee buffer.

- #define **FEE_NXT_JOB_CS_COPY_DATA_WRITE** (Fee_JobType)18
  Write data from internal Fee buffer to target chunk.

- #define **FEE_NXT_JOB_CS_CHUNK_HDR_VLD_DONE** (Fee_JobType)19
  Finalize format of first Fee chunk.

- #define **FEE_NXT_JOB_DONE** (Fee_JobType)20
  No more subsequent jobs to queue.

- #define **FEE_BLOCK_VALID** (Fee_BlockStatusType)0
  Fee block is valid.

- #define **FEE_BLOCK_INVALID** (Fee_BlockStatusType)1
  Fee block is invalid (has been invalidated).

- #define **FEE_BLOCK_INCONSISTENT** (Fee_BlockStatusType)2
  Fee block is inconsistent (contains bogus data).

- #define **FEE_BLOCK_HEADER_INVALID** (Fee_BlockStatusType)3
  Fee block header is garbled.

- #define **FEE_BLOCK_INVALIDATED** (Fee_BlockStatusType)4
  Fee block header is invalidated by Fee_InvalidateBlock(BlockNumber) (not used when FEE_BLOCK_ALWAYS_AVAILABLE == STD_OFF).

- #define **FEE_BLOCK_HEADER_BLANK** (Fee_BlockStatusType)5
  Fee block header is blank.

- #define **FEE_BLOCK_INCONSISTENT_COPY** (Fee_BlockStatusType)6
  FEE data read error during chunk switch (ie data area was allocated).

- #define **FEE_BLOCK_NEVER_WRITTEN** (Fee_BlockStatusType)7
  FEE block was never written in data flash.

- #define **FEE_CHUNK_VALID** (Fee_BlockStatusType)0
  Fee chunk is valid.

- #define **FEE_CHUNK_INVALID** (Fee_BlockStatusType)1
  Fee chunk is invalid.

- #define **FEE_CHUNK_INCONSISTENT** (Fee_BlockStatusType)2
  Fee chunk is inconsistent (contains error data).

- #define **FEE_CHUNK_HEADER_INVALID** (Fee_BlockStatusType)3
  Fee chunk header is garbled.

2.3.4 Macros in Fee_Types.h

- #define **FEE_PROJECT_SHARED** (Fee_BlockAssignmentType)1

- #define **FEE_PROJECT_APPLICATION** (Fee_BlockAssignmentType)2

- #define **FEE_PROJECT_BOOTLOADER** (Fee_BlockAssignmentType)3

- #define **FEE_PROJECT_RESERVED** (Fee_BlockAssignmentType)0xFF

2.3.5 Macros in Fee_version.h

- #define **FEE_AR_RELEASE_MAJOR_VERSION** 4
- #define **FEE_AR_RELEASE_MINOR_VERSION** 6
- #define **FEE_AR_RELEASE_REVISION_VERSION** 0
- #define **FEE_SW_MAJOR_VERSION** 1
- #define **FEE_SW_MINOR_VERSION** 2
- #define **FEE_SW_PATCH_VERSION** 0
- #define **FEE_VENDOR_ID** 174
- #define **FEE_MODULE_ID** 21

2.4 Enums

None

2.5 Typedefs

2.5.1 Typedefs in Fee_Internal.h

- typedef MemIf_JobResultType (***FeeJobQueMngrHandler**) (void)
  Fee job queue list handler.

2.5.2 Typedefs in Fee_InternalTypes.h

- typedef uint8 **Fee_JobType**
  This type represents the next job of the FEE module.

- typedef uint8 **Fee_BlockStatusType**
  Status of Fee block header.

- typedef uint8 **Fee_ChunkStatusType**
  Status of Fee chunk header.

2.5.3 Typedefs in Fee_Types.h

- typedef uint8 **Fee_BlockAssignmentType**
  Fee block assignment type.

- typedef Fee_BlockConfigType **Fee_ConfigType**

Fee Configuration type is a stub type, not used, but required by AUTOSAR.

2.6 Structures

2.6.1 Fee_ChunkGroupInfoType

Structure	Fee_ChunkGroupInfoType
Description	Fee chunk group run-time status.
Diagram	N/A
Data Fields	• Fls_AddressType u32DataAddrIt Address of current Fee data block in flash. • Fls_AddressType u32HdrAddrIt Address of current Fee block header in flash. • uint32 u32ActChunkID ID of active chunk. • uint8 u8ActChunk Index of active chunk.

2.6.2 Fee_BlockInfoType

Structure	Fee_BlockInfoType
Description	Fee block run-time status.
Diagram	N/A
Data Fields	• Fls_AddressType u32DataAddr Address of Fee block data in flash. More... • Fls_AddressType u32InvalidAddr Address of Fee block invalidation field in flash. More... • Fee_BlockStatusType u8BlockStatus Current status of Fee block.

2.6.3 Fee_tGlobalVar

Structure	Fee_tGlobalVar
Description	Structure containing the global variables of the module.
Diagram	N/A
Data Fields	• uint8 * pDataBufferPtr Pointer to data buffer, Used by read,write and swap jobs. • boolean bNeedToSwap Indicates whether a swap is needed. • uint8 * pDataReadDestPtr Pointer to destination buffer for Fee_Read() • const uint8 * pDataWriteDestPtr Pointer to source buffer for Fee_Write() • uint16 u16BlockIndex Fee block index. Used by all Fee jobs. • Fls_LengthType u32BlockOffset Fee block Offset. Used by the read Fee job. • Fls_LengthType u32BlockLength Fee block length. Used by the write Fee job. • Fee_JobType u8Job Current job. • Fee_JobType u8OriginalJob Original job (before any swap)

	• MemIf_StatusType eModuleStatus Current module status. • MemIf_JobResultType eJobResult Current job result. • uint8 u8ChunkGroupIt Internal iterator of chunk group. • uint8 u8ChunkIt Internal iterator of chunk. • uint16 u16BlockIt Internal iterator of Fee block. • Fls_AddressType u32IntAddrIt Internal flash helper address iterator. Used by the scan and swap jobs. • Fls_AddressType u32IntHdrAddr Internal address of current block header. Used by the swap job. • Fls_AddressType u32IntDataAddr Internal address of current data block. Used by the swap job. • Fee_BlockInfoType tBlockInfo [FEE_MAX_NUM_OF_BLOCKS_IN_FUTURE] Run-time status of all Fee blocks. • Fee_ChunkGroupInfoType tChunkGrpInfo [FEE_NUM_OF_CHUNK_GROUPS] Run-time status of all Fee chunk groups.

2.6.4 Fee_tPotentialGlobalVar

Structure	Fee_tPotentialGlobalVar
Description	Structure containing the potential global variables of the module.
Diagram	N/A
Data Fields	• uint16 u16ForeignBlocksNumber Used to keep the number of foreign blocks found when parsing the data flash. It represents the number of elements from the array. • Fee_BlockConfigType tForeignBlockConfig [FEE_MAX_NUM_OF_BLOCKS_IN_FUTURE - FEE_CFG_NUM_OF_BLOCKS] Used to keep the config of the foreign blocks. • uint32 au32ReservedAreaTouched [(FEE_MAX_NUM_OF_BLOCKS_IN_FUTURE+((sizeof(uint32) *8U) -1U))/(sizeof(uint32) *8U)] Run-time information about blocks touching the Reserved Area.

2.6.5 Fee_BlockConfigType

Structure	Fee_BlockConfigType
Description	Fee block configuration structure
Diagram	N/A
Data Fields	• uint16 u16BlockNumber Fee block number. More... • uint16 u16BlockSize Size of Fee block in bytes. More... • uint8 u8ChunkGrp Index of chunk group the Fee block belongs to. More... • boolean bImmediateData TRUE if immediate data block. More... • Fee_BlockAssignmentType u8BlockAssignment specifies which project uses this block

2.6.6 Fee_ChunkType

Structure	Fee_ChunkType
Description	Fee chunk configuration structure.
Diagram	N/A
Data Fields	• Fls_AddressType u32StartAddr Address of Fee chunk in flash. More... • Fls_LengthType u32Length Size of Fee chunk in bytes.

[SWDESG_FEE_097] The type imported from Fls module shall not be changed or extended for a specific FEE module or hardware platform.

2.6.7 Fee_ChunkGroupType

Structure	Fee_ChunkGroupType
Description	Fee chunk group configuration structure.
Diagram	N/A
Data Fields	• const Fee_ChunkType *const pChunkPtr Pointer to array of Fee chunk configurations. • uint32 u32ChunkCount Number of chunks in chunk group. • Fls_LengthType u32ResvSize Size of reserved area in the given chunk group (memory occupied by immediate blocks)

2.6.8 Fee_BlockType

Structure	Fee_BlockType
Description	Fee block header configuration structure. This consists of block number and length of block and Type of Fee block.
Diagram	N/A
Data Fields	• uint16 u16BlockNumber Number of block. • uint16 u16Length Length of block. • boolean bImmediateBlock Type of Fee block. Set to TRUE for immediate block.

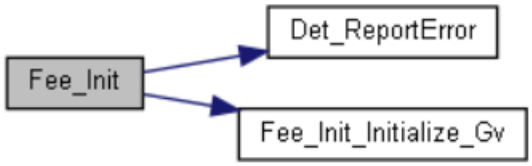
2.6.9 Fee_ChunkHeaderType

Structure	Fee_ChunkHeaderType
Description	Fee block header configuration structure. This consists of block number and length of block and Type of Fee block.
Diagram	N/A
Data Fields	• uint16 u16BlockNumber Number of block. • uint16 u16Length Length of block. • boolean bImmediateBlock Type of Fee block. Set to TRUE for immediate block.

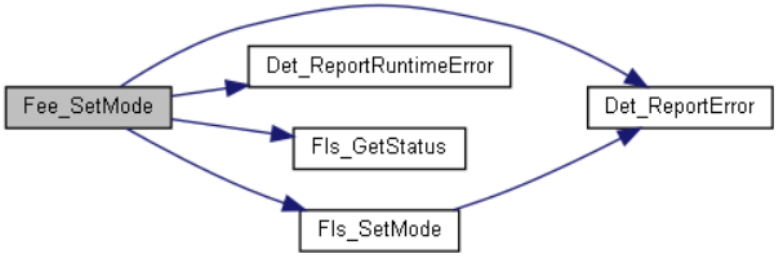
2.7 API Functions

2.7.1 Functions in Fee.h

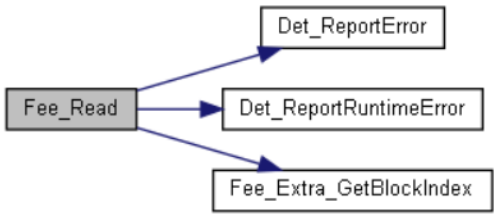
2.7.1.1 void Fee_Init(const Fee_ConfigType* ConfigPtr)

Function	void Fee_Init(const Fee_ConfigType* ConfigPtr)	
Description	Service to initialize the FEE module	
Diagram		
Parameters	**Parameter**	**Description**
	ConfigPtr	Pointer to Configuration data structure of the Fee module
Register Accessed	N/A	
Returns	void	

2.7.1.2 void Fee_SetMode(MemIf_ModeType Mode)

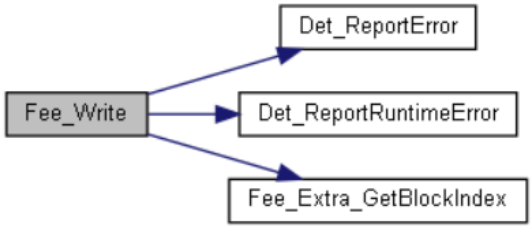
Function	void Fee_SetMode(MemIf_ModeType Mode)	
Description	Set the Fee module' s operation mode to the given Mode.	
Diagram		
Parameters	**Parameter**	**Description**
	Mode	MEMIF_MODE_FAST or MEMIF_MODE_SLOW
Register Accessed	N/A	
Returns	void	

2.7.1.3 Std_ReturnType Fee_Read(uint16 BlockNumber, uint16 BlockOffset, uint8* DataBufferPtr, uint16 Length)

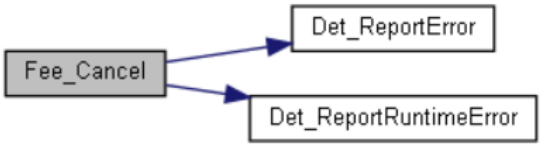
Function	Std_ReturnType Fee_Read(uint16 BlockNumber, uint16 BlockOffset, uint8* DataBufferPtr, uint16 Length)	
Description	Service to initiate a read job.	
Diagram		
Parameters	**Parameter**	**Description**
	BlockNumber	Number of logical block, also denoting start address of that block in flash memory.
	BlockOffset	Read address offset inside the block.
	DataBufferPtr	Pointer to data buffer.
	Length	Number of bytes to read.
Register Accessed	N/A	

Returns	E_OK	The read job was accepted by the underlying memory driver.
	E_NOT_OK	The read job has not been accepted by the underlying memory driver.

2.7.1.4 Std_ReturnType Fee_Write(uint16 BlockNumber, const uint8* DataBufferPtr)

Function	Std_ReturnType Fee_Write(uint16 BlockNumber, const uint8* DataBufferPtr)	
Description	Service to initiate a write job.	
Diagram		
Parameters	**Parameter**	**Description**
	BlockNumber	Number of logical blocks, also denoting start address of that block in emulated EEPROM.
	DataBufferPtr	Pointer to data buffer.
Register Accessed	N/A	
Returns	E_OK	The write job was accepted by the underlying memory driver.
	E_NOT_OK	The write job has not been accepted by the underlying memory driver.

2.7.1.5 void Fee_Cancel(void);

Function	void Fee_Cancel(void)	
Description	Service to call the cancel function of the underlying flash driver.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	

2.7.1.6 MemIf_StatusType Fee_GetStatus(void)

Function	MemIf_StatusType Fee_GetStatus(void)	
Description	Return the Fee module state.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MEMIF_UNINIT	Module has not been initialized.
	MEMIF_IDLE	Module is currently idle.
	MEMIF_BUSY	Module is currently busy.
	MEMIF_BUSY_INTERNAL	Module is busy with internal management operations.

2.7.1.7 MemIf_JobResultType Fee_GetJobResult(void)

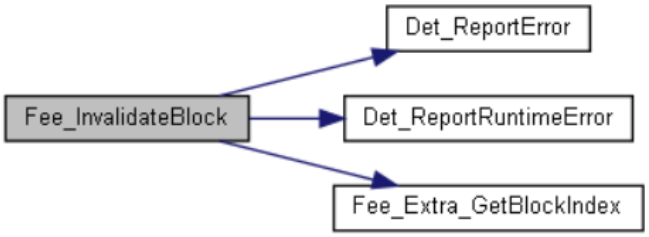
Function	MemIf_JobResultType Fee_GetJobResult(void)
Description	Return the result of the last job.
Diagram	

Parameters	Parameter	Description
	N/A	N/A
Register Accessed	N/A	
Returns	MEMIF_JOB_OK The job has been finished successfully. MEMIF_JOB_FAILED The job has not been finished successfully. MEMIF_JOB_PENDING The job has not yet been finished. MEMIF_JOB_CANCELED The job has been canceled. MEMIF_BLOCK_INCONSISTENT The requested block is inconsistent, it may contain corrupted data. MEMIF_BLOCK_INVALID The requested block has been invalidated, the requested read operation can not be performed.	

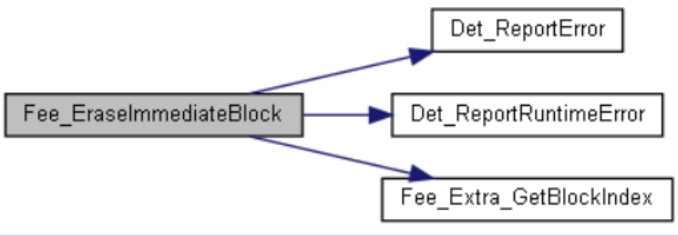
2.7.1.8 void Fee_GetVersionInfo(Std_VersionInfoType* VersionInfoPtr)

Function	void Fee_GetVersionInfo(Std_VersionInfoType* VersionInfoPtr)	
Description	Return the version information of the Fee module.	
Diagram		
Parameters	**Parameter**	**Description**
	VersionInfoPtr	Pointer to where to store the version information of this module.
Register Accessed	N/A	
Returns	N/A	

2.7.1.9 Std_ReturnType Fee_InvalidateBlock(uint16 BlockNumber)

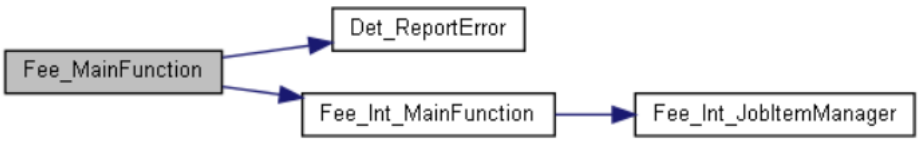
Function	Std_ReturnType Fee_InvalidateBlock(uint16 BlockNumber)	
Description	Service to invalidate a logical block.	
Diagram		
Parameters	**Parameter**	**Description**
	BlockNumber	Number of logical block, also denoting start address of that block in flash memory
Register Accessed	N/A	
Returns	E_OK The write job was accepted by the underlying memory driver. E_NOT_OK The write job has not been accepted by the underlying memory driver.	

2.7.1.10 Std_ReturnType Fee_EraseImmediateBlock(uint16 BlockNumber)

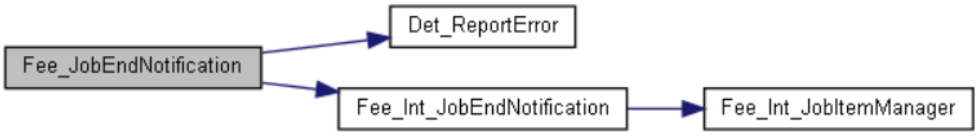
Function	Std_ReturnType Fee_EraseImmediateBlock(uint16 BlockNumber)	
Description	Service to erase a logical block.	
Diagram		
Parameters	**Parameter**	**Description**

	BlockNumber	Number of logical block, also denoting.
Register Accessed	N/A	
Returns	E_OK The write job was accepted by the underlying memory driver. E_NOT_OK The write job has not been accepted by the underlying memory driver.	

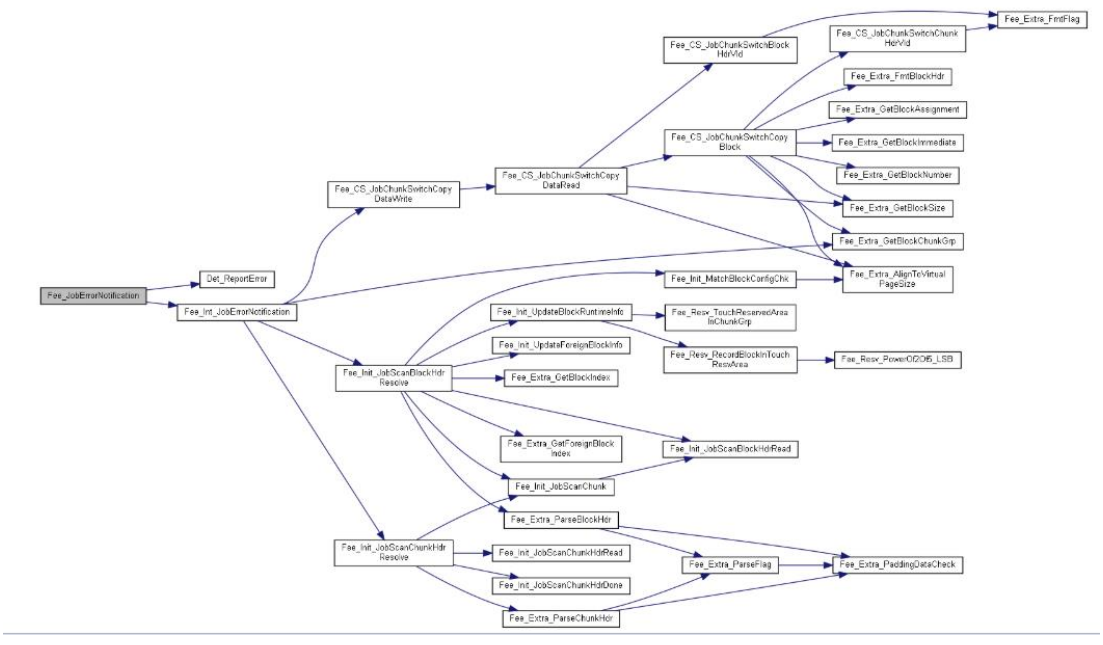
2.7.1.11 void Fee_MainFunction(void)

Function	void Fee_MainFunction(void)	
Description	Service to handle the requested read / write / erase jobs respectively the internal management operations.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	

2.7.1.12 void Fee_JobEndNotification(void)

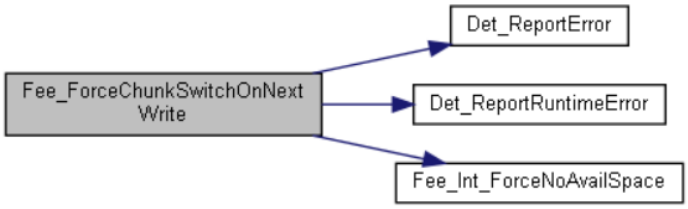
Function	void Fee_JobEndNotification(void)	
Description	Service to report the FEE module the successful end of an asynchronous operation.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	

2.7.1.13 void Fee_JobErrorNotification(void)

Function	void Fee_JobErrorNotification(void)
Description	Service to report the FEE module the failure of an asynchronous operation.
Diagram	

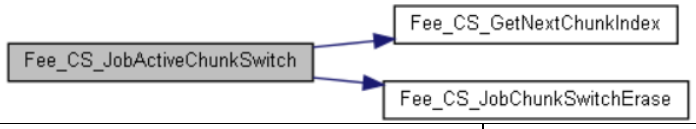
Parameters	Parameter	Description
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	

2.7.1.14 Std_ReturnType Fee_ForceChunkSwitchOnNextWrite(uint8 u8ChunkGrpIndex)

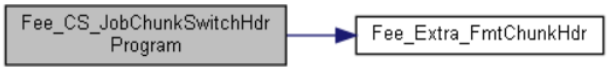
Function	Std_ReturnType Fee_ForceChunkSwitchOnNextWrite(uint8 u8ChunkGrpIndex)	
Description	Service to prepare the driver for a chunk switch in the selected chunk group.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	E_OK The job was accepted by the underlying memory driver. E_NOT_OK The job has not been accepted by the underlying memory driver.	

2.7.2 Functions in Fee_ChunkSwitch.h

2.7.2.1 MemIf_JobResultType Fee_CS_JobActiveChunkSwitch(void)

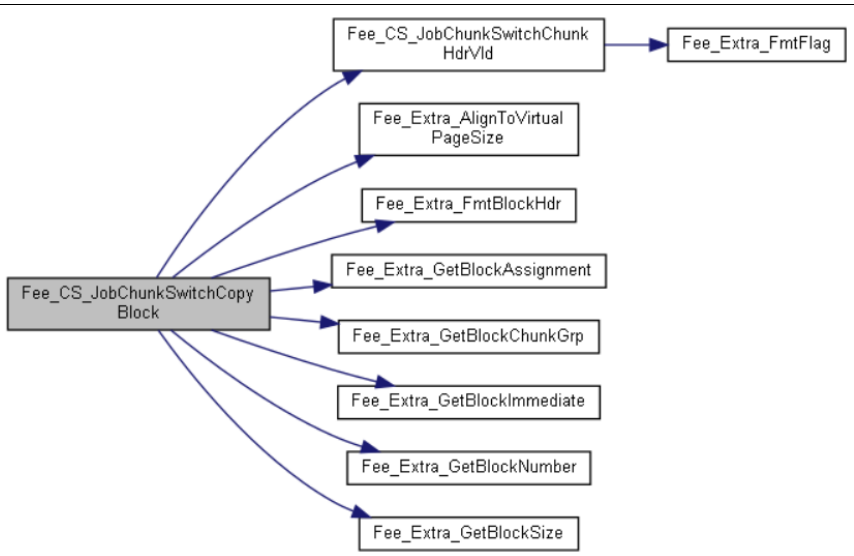
Function	MemIf_JobResultType Fee_CS_JobActiveChunkSwitch(void)	
Description	Searches ordered list of Fee blocks and returns index of block with matching BlockNumber.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_EraseImmediateBlock(), Fee_Int_JobWriteBlock().	

2.7.2.2 MemIf_JobResultType Fee_CS_JobChunkSwitchHdrProgram(void)

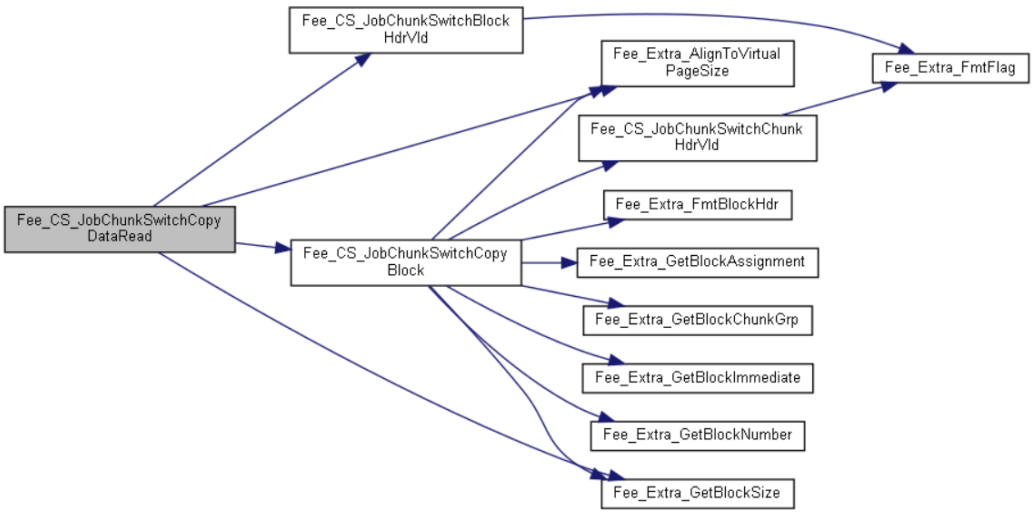
Function	MemIf_JobResultType Fee_CS_JobChunkSwitchHdrProgram(void)	
Description	Program chunk header in current chunk(to be switched) because of recent erase	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable	

2.7.2.3 MemIf_JobResultType Fee_CS_JobChunkSwitchCopyBlock(void)

Function	MemIf_JobResultType Fee_CS_JobChunkSwitchCopyBlock(void)
Description	Copy block from source to target chunk

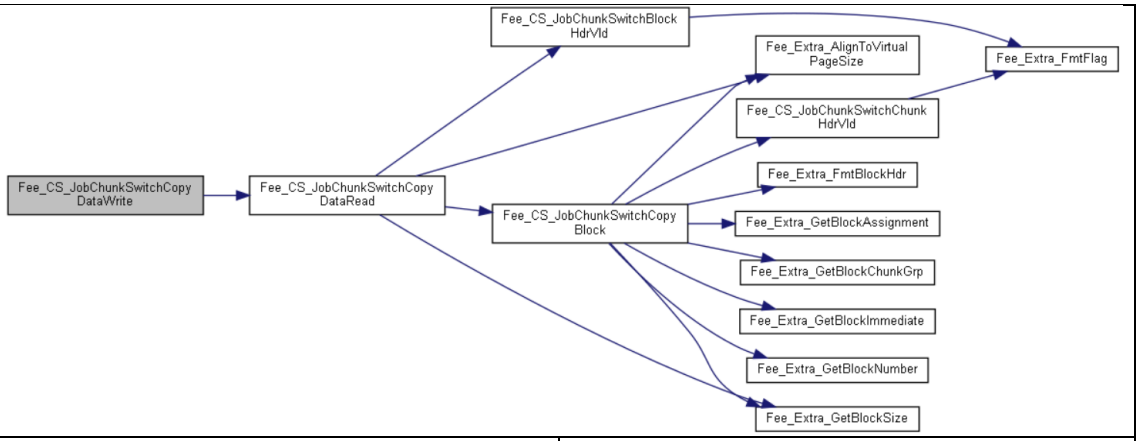
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable, Fee_CS_JobChunkSwitchCopyDataRead	

2.7.2.4 MemIf_JobResultType Fee_CS_JobChunkSwitchCopyDataRead (const boolean bStatus))

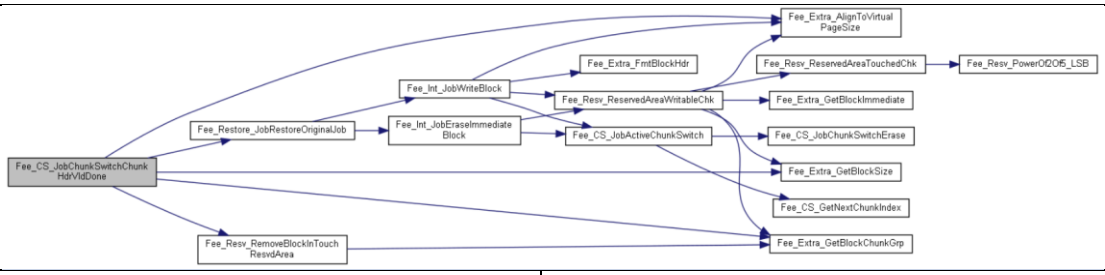
Function	MemIf_JobResultType Fee_CS_JobChunkSwitchCopyDataRead (const boolean bStatus))	
Description	Read data from source chunk to internal data buffer	
Diagram		
Parameters	**Parameter**	**Description**
	bStatus	FALSE if previous Fls read job has failed
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_CS_JobChunkSwitchCopyDataWrite	

2.7.2.5 MemIf_JobResultType Fee_CS_JobChunkSwitchCopyDataWrite (const boolean bStatus)

Function	MemIf_JobResultType Fee_CS_JobChunkSwitchCopyDataWrite (const boolean bStatus)
Description	Write data from data buffer to target chunk

Diagram		
Parameters	**Parameter**	**Description**
	bStatus	FALSE if previous Fls read job has failed
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Int_JobErrorNotification	

2.7.2.6 MemIf_JobResultType Fee_CS_JobChunkSwitchChunkHdrVldDone (void)

Function	MemIf_JobResultType Fee_CS_JobChunkSwitchCopyDataWrite (const boolean bStatus)	
Description	Program chunk Header when chunk switch is done	
Diagram		
Parameters	**Parameter**	**Description**
	bStatus	FALSE if previous Fls read job has failed
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable	

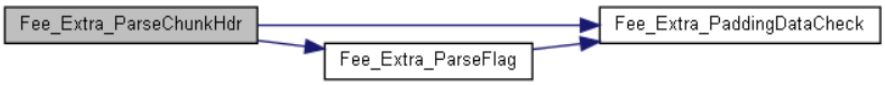
2.7.3 Functions in Fee_Extra.h

2.7.3.1 void Fee_Extra_FmtChunkHdr (const Fee_ChunkHeaderType *pChunkHdrPtr, uint8 *pDataBufPtr)

Function	void Fee_Extra_FmtChunkHdr (const Fee_ChunkHeaderType *pChunkHdrPtr, uint8 *pDataBufPtr)	
Description	Format Fee chunk header parameters to write buffer	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	pChunkHdrPtr	Chunk header type
	pDataBufPtr	Pointer to write buffer
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_CS_JobChunkSwitchHdrProgram, Fee_Init_JobScanChunkProgram	

2.7.3.2 Fee_ChunkStatusType Fee_Extra_ParseChunkHdr (Fee_ChunkHeaderType *pChunkHdr, const uint8 *pDataPtr)

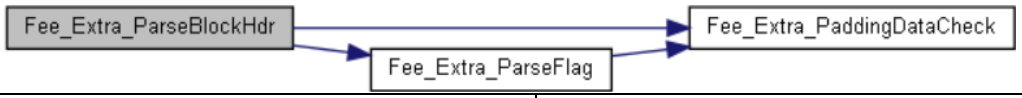
Function	Fee_ChunkStatusType Fee_Extra_ParseChunkHdr(Fee_ChunkHeaderType*pChunkHdr,const uint8 *pDataPtr)
Description	Parse Fee chunk header parameters from read buffer

Diagram		
Parameters	**Parameter**	**Description**
	pChunkHdrPtr	Pointer to chunk header
	pDataBufPtr	Pointer to read buffer
Register Accessed	N/A	
Returns	Fee_ChunkStatusType	
Referenced By	Fee_Init_JobScanChunkHdrResolve	

2.7.3.3 void Fee_Extra_FmtBlockHdr (const Fee_BlockType *tBlockHdr, const Fls_AddressType u32TargetAddress, const Fee_BlockAssignmentType u8BlockAssignment, uint8 *pDataBuf)

Function	void Fee_Extra_FmtBlockHdr (const Fee_BlockType *tBlockHdr, const Fls_AddressType u32TargetAddress, const Fee_BlockAssignmentType u8BlockAssignment, uint8 *pDataBuf)	
Description	Format Fee block parameters into a write buffer	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	tBlockHdr	block header (block number and Length)
	u32TargetAddress	Logical address of Fee block in Fls adress space
	u8BlockAssignment	Block assignment
	pDataBuf	Pointer to format buffer
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_CS_JobChunkSwitchCopyBlock, Fee_Int_JobWriteBlock	

2.7.3.4 Fee_BlockStatusType Fee_Extra_ParseBlockHdr (Fee_BlockType *const pBlockHdrPtr, Fls_AddressType *const pDataAddrPtr, uint8 *const pBlockAssignmentPtr, const uint8 *pDataBufPtr)

Function	Fee_BlockStatusType Fee_Extra_ParseBlockHdr (Fee_BlockType *const pBlockHdrPtr, Fls_AddressType *const pDataAddrPtr, uint8 *const pBlockAssignmentPtr, const uint8 *pDataBufPtr)	
Description	Parse Fee block header parameters from read buffer.	
Diagram		
Parameters	**Parameter**	**Description**
	pBlockHdrPtr	Fee Block Header (block number and Length)
	pDataAddrPtr	Pointer to fee block data address
	pBlockAssignmentPtr	Pointer to fee block assignment(optional)
	pDataBufPtr	Pointer to read buffer
Register Accessed	N/A	
Returns	Fee_BlockStatusType	
Referenced By	Fee_Init_JobScanBlockHdrResolve	

2.7.3.5 void Fee_Extra_FmtFlag (uint8 *pTargetPtr, const uint8 u8FlagMode)

Function	void Fee_Extra_FmtFlag (uint8 *pTargetPtr, const uint8 u8FlagMode)	
Description	Format validation or invalidation flag to write buffer.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	pTargetPtr	Pointer to write buffer
	u8FlagMode	FEE_VALIDATED_VALUE or FEE_INVALIDATED_VALUE

Register Accessed	N/A
Returns	N/A
Referenced By	Fee_CS_JobChunkSwitchBlockHdrVld, Fee_CS_JobChunkSwitchChunkHdrVld, Fee_Init_JobScanChunkProgram, Fee_Int_JobValidBlockHdr, Fee_Int_JobInvalidateBlock

2.7.3.6 Std_ReturnType Fee_Extra_PaddingDataCheck (const uint8 *pTargetPtr, const uint8 *const pTargetEndPtr)

Function	Std_ReturnType Fee_Extra_PaddingDataCheck(const uint8 *pTargetPtr, const uint8 *const pTargetEndPtr)	
Description	Check whether specified data buffer contains only the FEE_ERASED_VALUE value.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	pTargetPtr	Pointer to write buffer
	pTargetEndPtr	pointer to end + 1 of the checked buffer
Register Accessed	N/A	
Returns	Std_ReturnType	
Referenced By	Fee_Extra_ParseChunkHdr, Fee_Extra_ParseBlockHdr, Fee_Extra_ParseFlag	

2.7.3.7 Std_ReturnType Fee_Extra_ParseFlag (const uint8 *const pTargetPtr, const uint8 u8FlagMode, boolean *pFlagVal)

Function	Std_ReturnType Fee_Extra_ParseFlag (const uint8 *const pTargetPtr, const uint8 u8FlagMode, boolean *pFlagVal)	
Description	Parse the valid or invalid flag from a read buffer.	
Diagram		
Parameters	**Parameter**	**Description**
	pTargetPtr	Pointer to write buffer
	u8FlagMode	FEE_VALIDATED_VALUE or FEE_INVALIDATED_VALUE
	pFlagVal	TRUE if flag of above type is set
Register Accessed	N/A	
Returns	Std_ReturnType	
Referenced By	Fee_Extra_ParseChunkHdr, Fee_Extra_ParseBlockHdr	

2.7.3.8 uint16 Fee_Extra_GetBlockIndex (const uint16 u16BlockNumber)

Function	uint16 Fee_Extra_GetBlockIndex (const uint16 u16BlockNumber)	
Description	Searches ordered list of Fee blocks and returns index of block with matching BlockNumber.	
Diagram		
Parameters	**Parameter**	**Description**
	u16BlockNumber	Fee block number (FeeBlockNumber)
Register Accessed	N/A	
Returns	uint16 block index	
Referenced By	Fee_Init_JobScanBlockHdrResolve, Fee_Read, Fee_Write, Fee_InvalidateBlock, Fee_EraseImmediateBlock	

2.7.3.9 uint16 Fee_Extra_AlignToVirtualPageSize (uint16 u16BlockSize)

Function	uint16 Fee_Extra_AlignToVirtualPageSize (uint16 u16BlockSize)	
Description	Align block size to multiple of FEE_VIRTUAL_PAGE_SIZE.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	u16BlockSize	Fee block size (FeeBlockSize)
Register Accessed	N/A	

Returns	uint16 aligned block size
Referenced By	Fee_CS_JobChunkSwitchCopyBlock, Fee_CS_JobChunkSwitchCopyDataRead, Fee_CS_JobChunkSwitchChunkHdrVldDone, Fee_Init_MatchBlockConfigChk, Fee_Int_JobWriteBlockData, Fee_Int_JobWriteBlock, Fee_Resv_ReservedAreaWritableChk

2.7.3.10 void Fee_Extra_CopyDataToBuffer (const uint8 *pSourcePtr, uint8 *pTargetPtr, const uint16 u16Length)

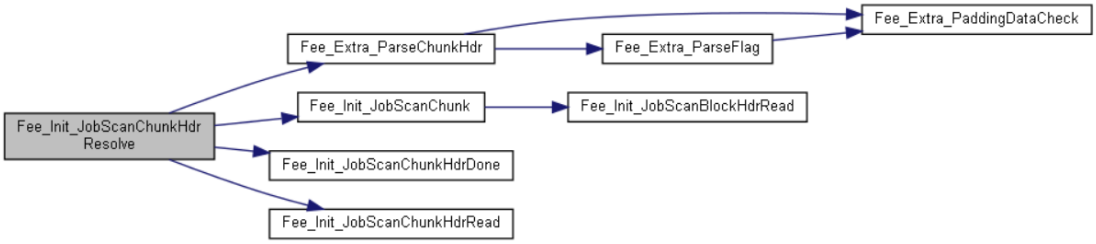
Function	void Fee_Extra_CopyDataToBuffer (const uint8 *pSourcePtr, uint8 *pTargetPtr, const uint16 u16Length)	
Description	Copy data from user to internal write buffer and fills rest of the write buffer with FEE_ERASED_VALUE.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	pSourcePtr	Pointer to user data buffer
	pTargetPtr	Pointer to internal write buffer
	u16Length	Number of bytes to copy
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_Int_JobWriteBlockData, Fee_Int_JobWriteBlockUnalignedData	

2.7.4 Functions in Fee_Initialization.h

2.7.4.1 void Fee_Init_Initialize_Gv (void)

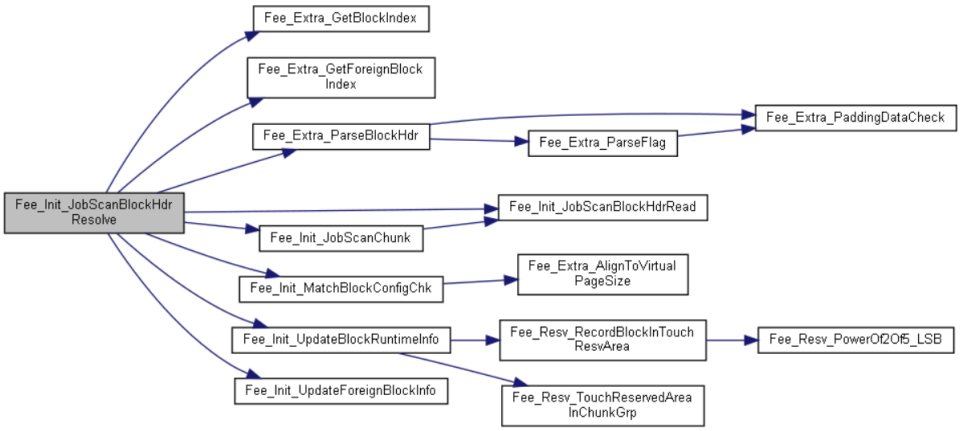
Function	void Fee_Init_Initialize_Gv (void)	
Description	Intial all global variables.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_Init	

2.7.4.2 MemIf_JobResultType Fee_Init_JobScanChunkHdrResolve(const boolean bStatus)

Function	MemIf_JobResultType Fee_Init_JobScanChunkHdrResolve(const boolean bStatus)	
Description	Resolve Fee chunk header.	
Diagram		
Parameters	**Parameter**	**Description**
	bStatus	FALSE if previous Fls read job has failed
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Int_JobErrorNotification	

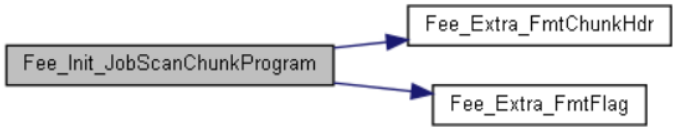
2.7.4.3 MemIf_JobResultType Fee_Init_JobScanBlockHdrResolve (const boolean bStatus)

Function	MemIf_JobResultType Fee_Init_JobScanBlockHdrResolve (const boolean bStatus)
Description	Resolve Fee block header.

Diagram		
Parameters	**Parameter**	**Description**
	bStatus	FALSE if previous Fls read job has failed
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Int_JobErrorNotification	

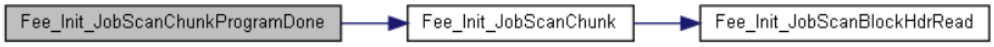
[SWDESG_FEE_094] Fee_Init_JobScanBlockHdrResolve shall resolve Fee block header.

2.7.4.4 MemIf_JobResultType Fee_Init_JobScanChunkProgram (void)

Function	MemIf_JobResultType Fee_Init_JobScanChunkProgram (void)	
Description	Program first Fee chunk in current Fee chunk group by writing chunk header into flash.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable	

[SWDESG_FEE_095] Fee_Init_JobScanChunkProgram shall program first Fee chunk in current Fee chunk group by writing chunk header into flash.

2.7.4.5 MemIf_JobResultType Fee_Init_JobScanChunkProgramDone (void)

Function	MemIf_JobResultType Fee_Init_JobScanChunkProgramDone (void)	
Description	Finalize program of first Fee chunk in current Fee chunk group.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable	

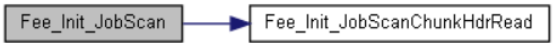
[SWDESG_FEE_096] Fee_Init_JobScanChunkProgramDone shall finalize program of first Fee chunk in current Fee chunk group.

2.7.4.6 MemIf_JobResultType Fee_Init_JobScanChunk (void)

Function	MemIf_JobResultType Fee_Init_JobScanChunk (void)
Description	Scan active chunk of current chunk group or erase and format first chunk if no active chunk can be found.

Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Init_JobScanChunkHdrResolve, Fee_Init_JobScanBlockHdrResolve, Fee_Init_JobScanChunkProgramDone, Fee_aJobFuncTable	

2.7.4.7 MemIf_JobResultType Fee_Init_JobScan (void)

Function	MemIf_JobResultType Fee_Init_JobScan (void)	
Description	Initialize the chunk scan job.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_aJobFuncTable	

2.7.5 Functions in Fee_Internal.h

2.7.5.1 void Fee_Int_MainFunction (void)

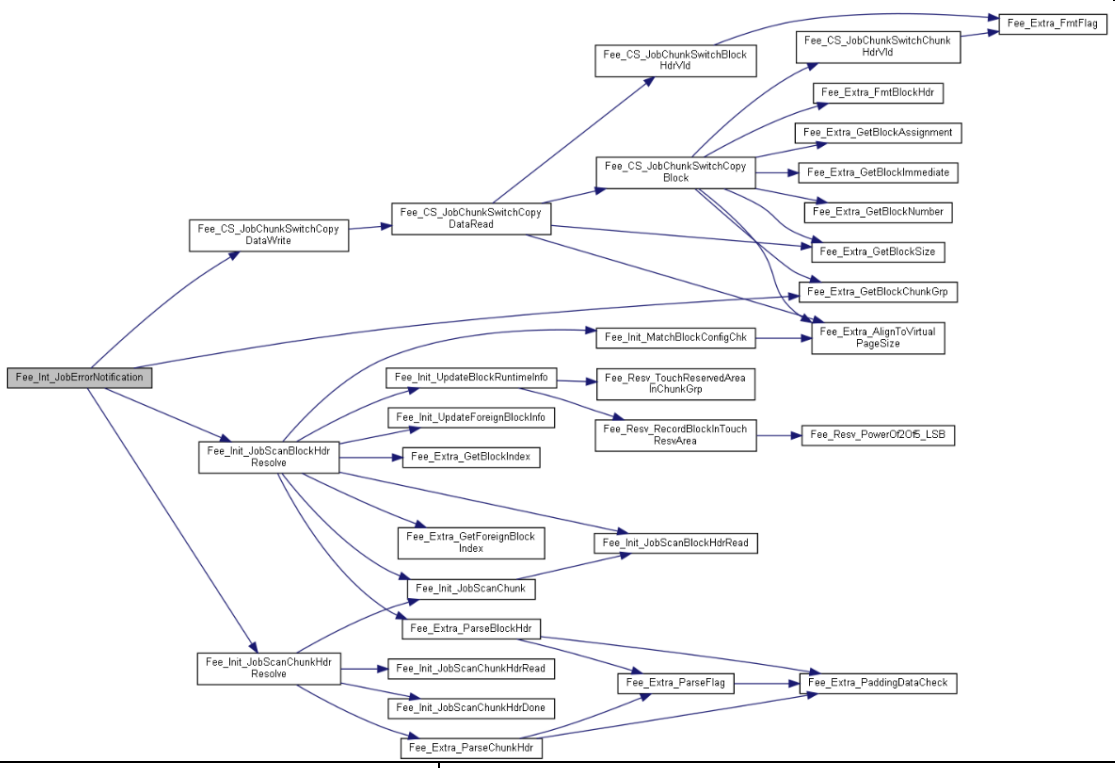
Function	void Fee_Int_MainFunction (void)	
Description	Fee mainfunction internal job function.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_MainFunction	

2.7.5.2 void Fee_Int_JobEndNotification (void)

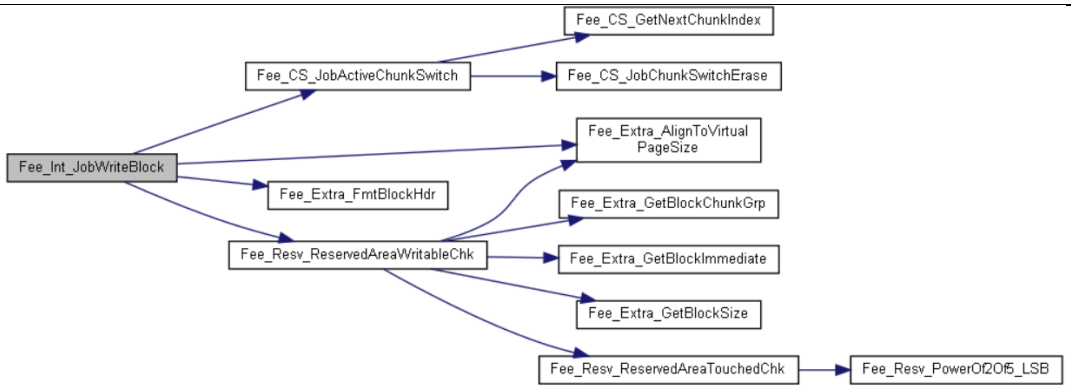
Function	void Fee_Int_JobEndNotification (void)	
Description	Fee job end notification internal job function.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_JobEndNotification	

2.7.5.3 void Fee_Int_JobErrorNotification (void)

Function	void Fee_Int_JobErrorNotification (void)
Description	Fee job error notification internal job function.

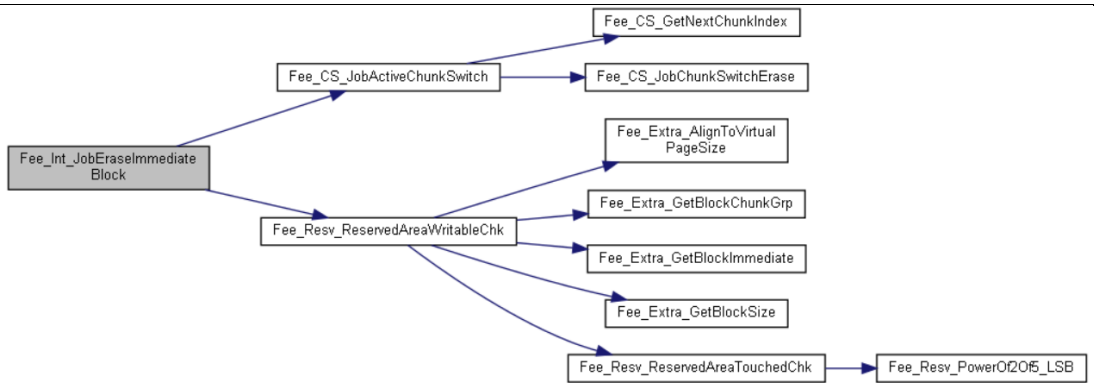
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_JobErrorNotification	

2.7.5.4 MemIf_JobResultType Fee_Int_JobWriteBlock (void)

Function	MemIf_JobResultType Fee_Int_JobWriteBlock (void)	
Description	Fee write block internal job function.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Restore_JobRestoreOriginalJob, Fee_aJobFuncTable	

2.7.5.5 MemIf_JobResultType Fee_Int_JobEraseImmediateBlock (void)

Function	MemIf_JobResultType Fee_Int_JobEraseImmediateBlock (void)
Description	Erase (pre-allocate) immediate Fee block.

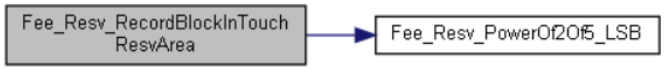
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_Restore_JobRestoreOriginalJob, Fee_aJobFuncTable	

2.7.6 Functions in Fee_Reserve.h

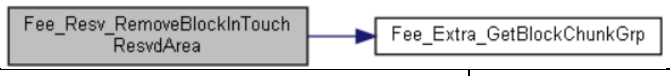
2.7.6.1 boolean Fee_Resv_TouchReservedAreaInChunkGrp (const uint8 u8ChunkGrpIndex)

Function	boolean Fee_Resv_TouchReservedAreaInChunkGrp (const uint8 u8ChunkGrpIndex)	
Description	Checks whether the area specified by header and data address touches the Reserved Area.	
Diagram	N/A	
Parameters	**Parameter**	**Description**
	u8ChunkGrpIndex	Chunk group index
Register Accessed	N/A	
Returns	Boolean job result	
Referenced By	Fee_Init_UpdateBlockRuntimeInfo, Fee_Int_JobWriteBlockData	

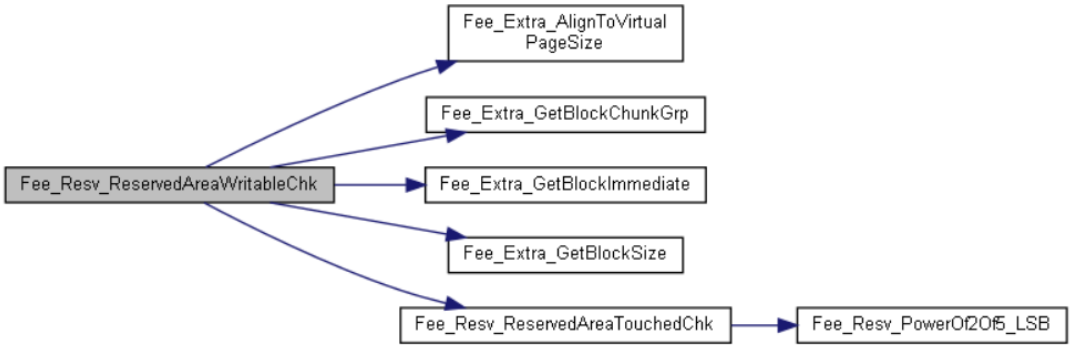
2.7.6.2 void Fee_Resv_RecordBlockInTouchResvArea (const uint16 u16BlockNumber)

Function	void Fee_Resv_RecordBlockInTouchResvArea (const uint16 u16BlockNumber)	
Description	Stores the information about touching the Reserved Area for the block specified by BlockNumber.	
Diagram		
Parameters	**Parameter**	**Description**
	u16BlockNumber	Block number
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_Int_JobWriteBlockData	

2.7.6.3 void Fee_Resv_RemoveBlockInTouchResvdArea (const uint8 u8ChunkGrpIndex)

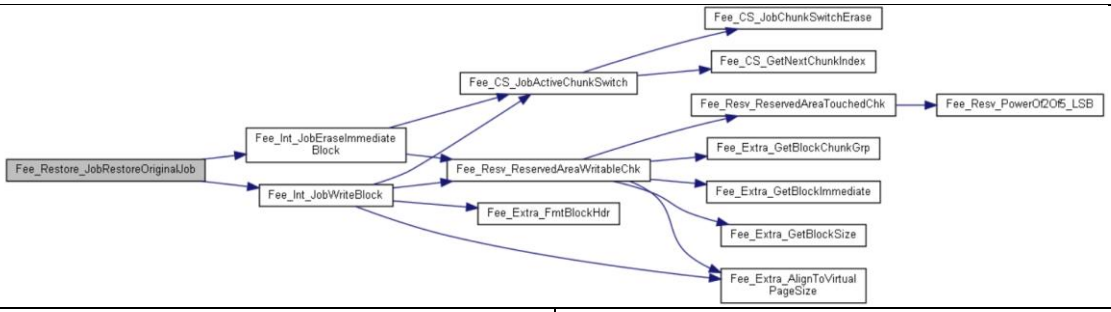
Function	void Fee_Resv_RemoveBlockInTouchResvdArea (const uint8 u8ChunkGrpIndex)	
Description	Removes the information about touching the Reserved Area for all blocks within a chunk group specified by u8ChunkGrpIndex.	
Diagram		
Parameters	**Parameter**	**Description**
	u8ChunkGrpIndex	Chunk group index
Register Accessed	N/A	
Returns	N/A	
Referenced By	Fee_CS_JobChunkSwitchChunkHdrVldDone	

2.7.6.4 boolean Fee_Resv_ReservedAreaWritableChk (void)

Function	boolean Fee_Resv_ReservedAreaWritableChk (void)	
Description	Checks whether the block is writable into the reserved area.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	Boolean job result	
Referenced By	Fee_Int_JobEraseImmediateBlock, Fee_Int_JobWriteBlock	

2.7.7 Functions in Fee_Restore.h

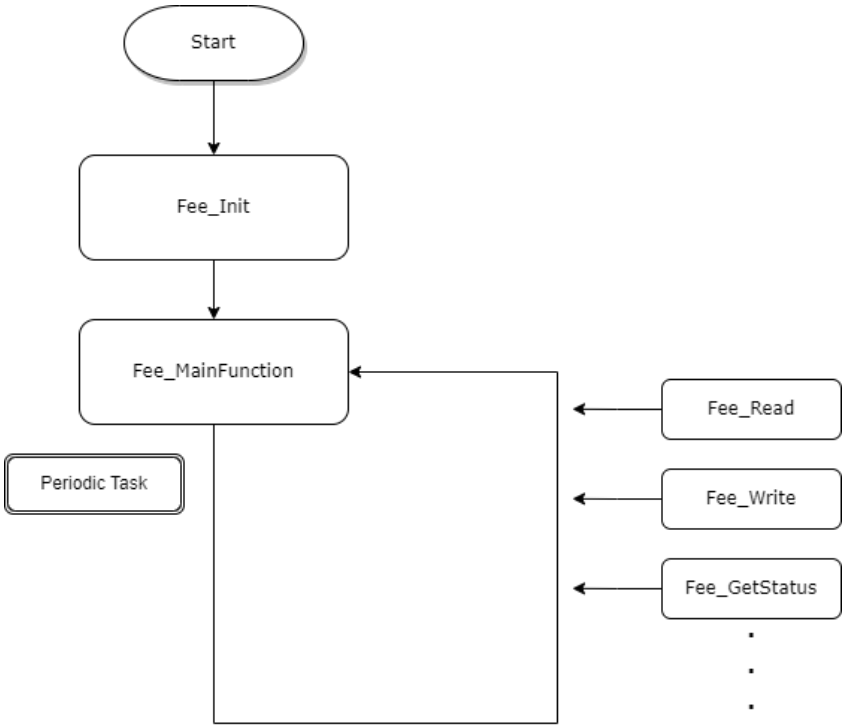
2.7.7.1 MemIf_JobResultType Fee_Restore_JobRestoreOriginalJob (void)

Function	MemIf_JobResultType Fee_Restore_JobRestoreOriginalJob (void)	
Description	Restore Original Job after chunk switch done.	
Diagram		
Parameters	**Parameter**	**Description**
	N/A	N/A
Register Accessed	N/A	
Returns	MemIf_JobResultType job result	
Referenced By	Fee_CS_JobChunkSwitchChunkHdrVldDone	

2.8 Software Architecture

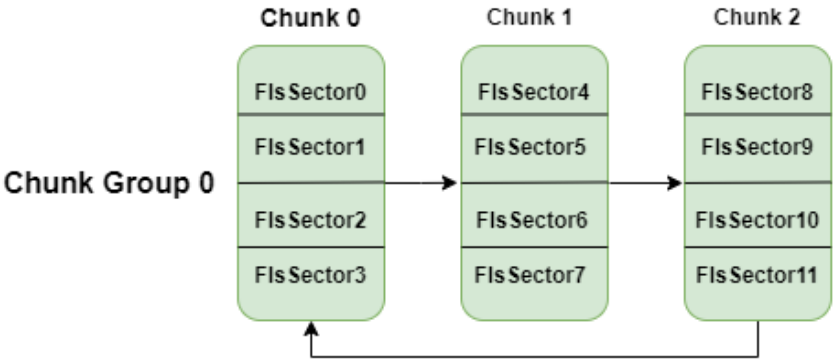
The FEE module is a pure software of memory management algorithm provides services for reading, writing, erasing and invalidating emulated EEPROM blocks apart from other basic features specified by the software specification

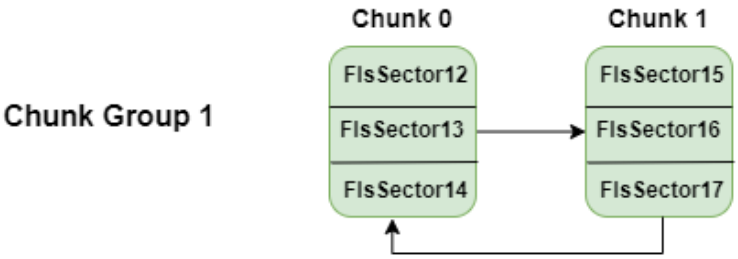
The following figure shows FEE software running process.

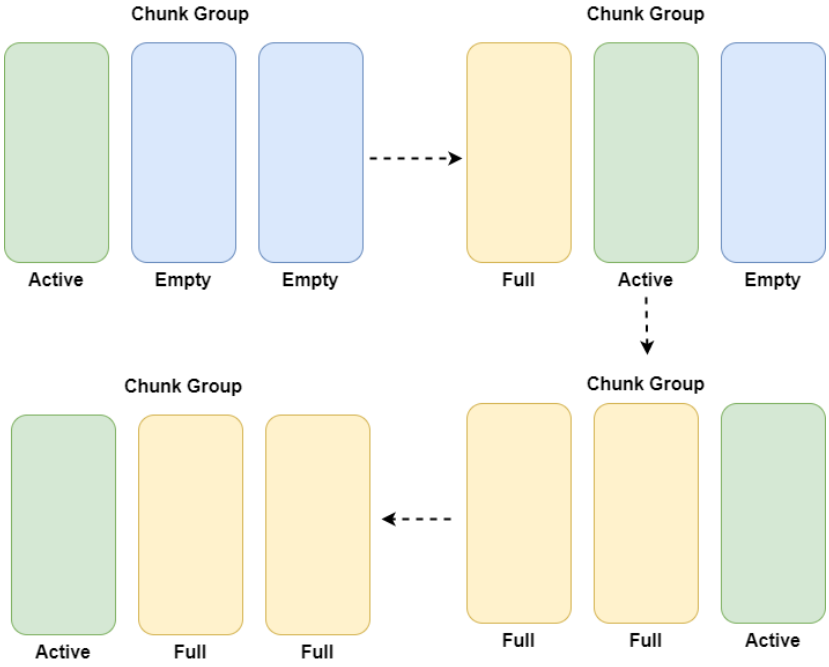


2.8.1 FEE Memory Organization

The Fee module will divide the memory into multiple unit groups based on the configuration, and each one is called chunk group. Every chunk group has multiple units, each unit called chunk and chunk is composed of actual FLS sectors, every chunk group containing 2 or more chunks for chunk switch.







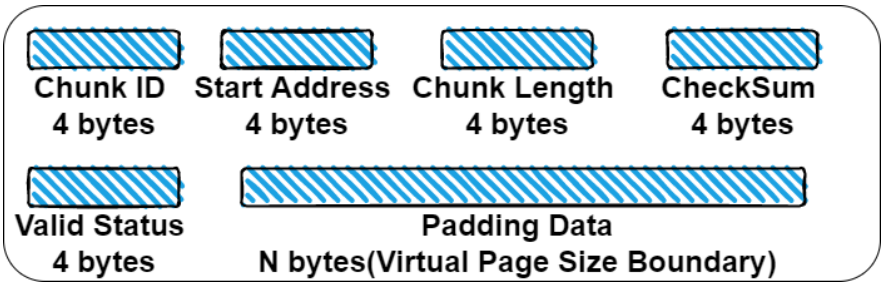
When the FEE module is configured (**FEE_VIRTUAL_PAGE_SIZE=8**), the data distribution in memory is as follows:

4 bytes		4 bytes	4 bytes	4 bytes	Description
Chunk ID		Start Address	Chunk Length	CheckSum	Chunk Header Status
Chunk Valid Status		Padding Data			
Block ID	Block Size	Block Data Address	CheckSum	Block Assignment(1byte)	Block Header Status
Block Valid Status		Padding Data	Block Invalid Status	Padding Data	
…					…
Block N Data					Block Data
Block N-1 Data					Block Data
…					Block Data
Block 2 Data					Block Data
Block 1 Data					Block Data

When the FEE module is configured (**FEE_VIRTUAL_PAGE_SIZE=16**), the data distribution in memory is as follows:

<table>
<tr><th colspan="2">4 bytes</th><th>4 bytes</th><th>4 bytes</th><th>4 bytes</th><th>Description</th></tr>
<tr><td colspan="2">Chunk ID</td><td>Start Address</td><td>Chunk Length</td><td>CheckSum</td><td rowspan="3">Chunk Header Status</td></tr>
<tr><td colspan="2">Chunk Valid Status</td><td colspan="3">Padding Data</td></tr>
<tr><td colspan="5">Padding Data</td></tr>
<tr><td>Block ID</td><td>Block Size</td><td>Block Data Address</td><td>CheckSum</td><td>Block Assignment(1byte)</td><td rowspan="3">Block Header Status</td></tr>
<tr><td colspan="2">Block Valid Status</td><td colspan="3">Padding Data</td></tr>
<tr><td colspan="2">Block Invalid Status</td><td colspan="3">Padding Data</td></tr>
<tr><td colspan="5">…
…
…
…
…</td><td>…</td></tr>
<tr><td colspan="5">Block N Data</td><td>Block Data</td></tr>
<tr><td colspan="5">Block N-1 Data</td><td>Block Data</td></tr>
<tr><td colspan="5">…</td><td>Block Data</td></tr>
<tr><td colspan="5">Block 2 Data</td><td>Block Data</td></tr>
<tr><td colspan="5">Block 1 Data</td><td>Block Data</td></tr>
</table>

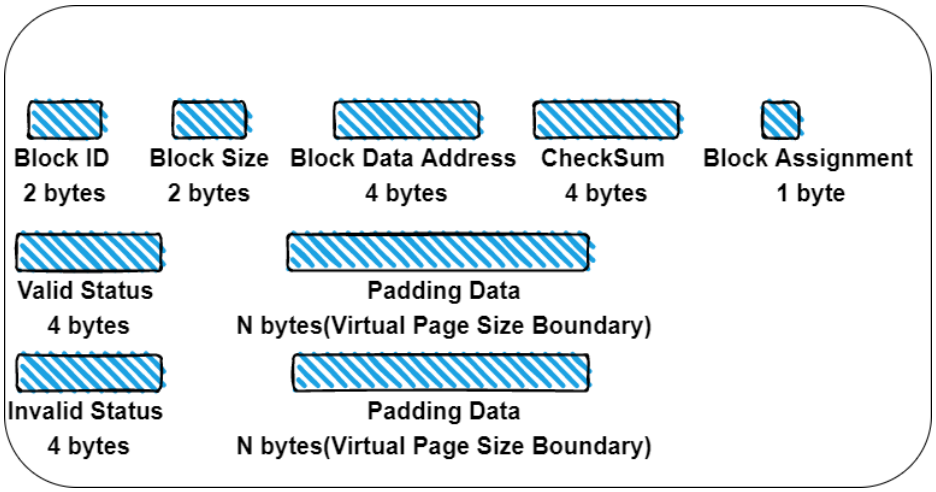
Structure of chunk header:



In which:

- **Chunk ID**: Integer number uniquely identifying the chunk. The number is incremented whenever a new chunk becomes active, i.e. the chunk with highest Chunk ID is the active one.
- **Start Address**: Start address of the chunk (Configuration data).
- **Chunk Length**: Length of chunk (Configuration data).
- **CheckSum**: Sum of the **Chunk ID**, **Start Address** and **Chunk Length**.
- **Valid Status**: Magic number 0x81 for valid, the field is padded with blank bytes (0xFF) to the virtual page size boundary.
- **Padding Data**: The field is padded with blank bytes (0xFF) to the virtual page size boundary.

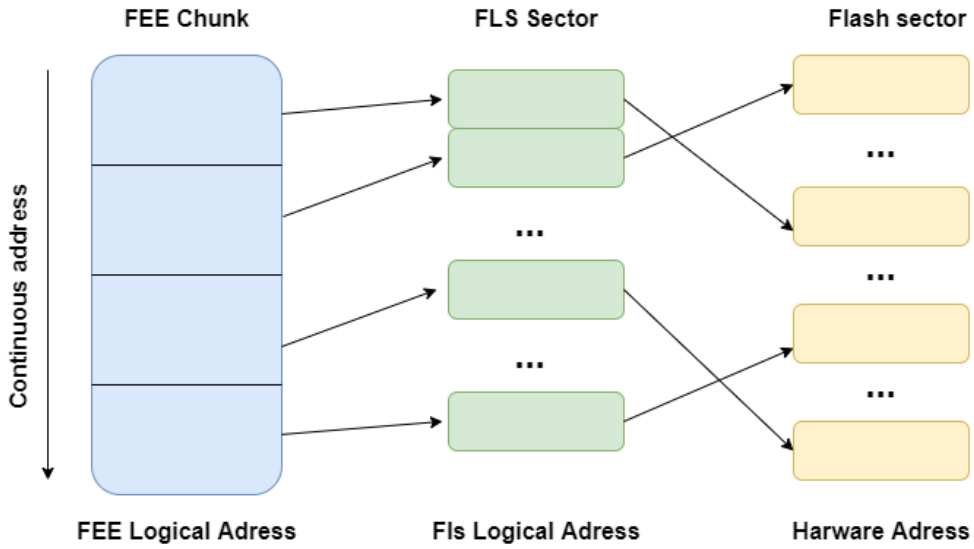
Structure of block header:



In which:

- **Block ID**: Integer number uniquely identifying the block (Configuration data).
- **Block Size**: Length of the block (Configuration data).
- **Block Data Address**: Logical address of the beginning of data area of this block.
- **CheckSum**: Sum of the **Block ID**, **Block Size** and **Block Data Address**.
- **Valid Status**: Magic number 0x81 for valid, the field is padded with blank bytes (0xFF) to the virtual page size boundary.
- **Padding Data**: The field is padded with blank bytes (0xFF) to the virtual page size boundary.
- **Invalid Status**: Magic number 0x18 for invalid (means the block was invalidated), the field is padded with blank bytes (0xFF) to the virtual page size boundary.
- **Padding Data**: The field is padded with blank bytes (0xFF) to the virtual page size boundary.

From the perspective of the FEE module, all emulated addresses are contiguous. However, the actual addresses written to the flash memory are those processed through address mapping. As a result, the data distribution in the flash memory does not necessarily reside in contiguous physical addresses. Nevertheless, it is certain that all data must fall within the configured address range.

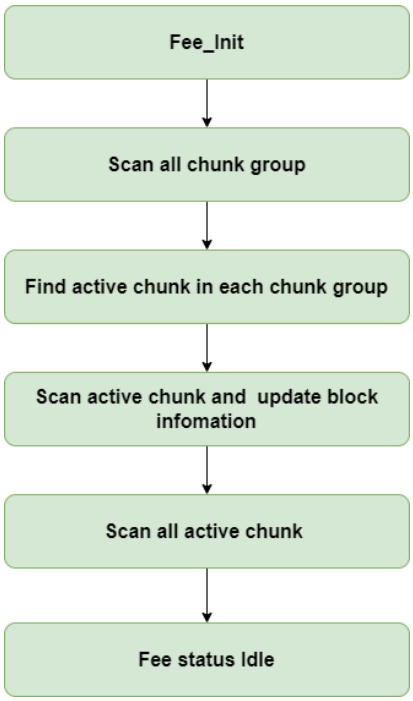


Although FEE-FLS supports configuration of non-contiguous physical addresses, it is strongly recommended that users configure contiguous physical addresses for easier management.

2.8.2 FEE Initialization

The FEE module is initialized after MCU power on, and it will process as follow:

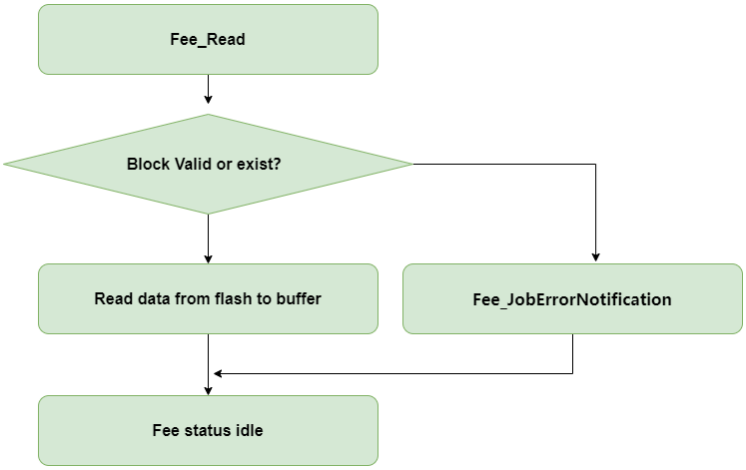
1. FEE start scan all chunk group through user configure information to find active chunk in current chunk group;
2. FEE scan active chunk which is found in step 1 for updating block information;
3. After scanning all active chunk FEE set module status to idle.



After the initialization of FEE module, all fee services can be process including reading, writing and so on.

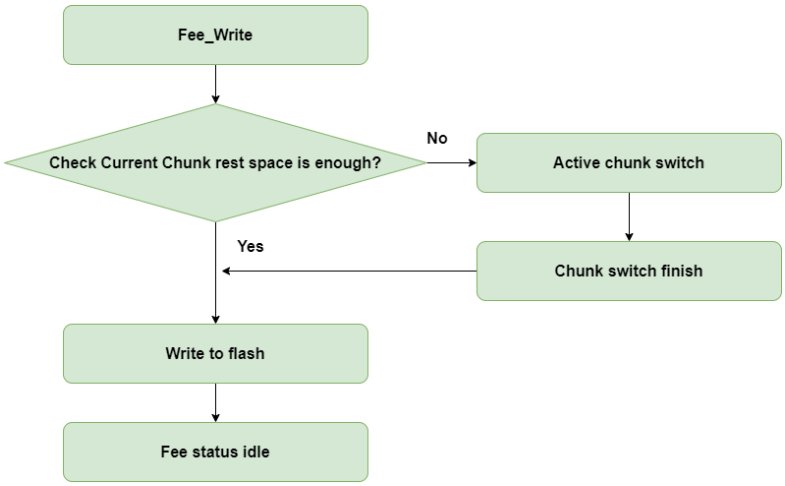
2.8.3 FEE Read

During an FEE read operation, the validity of the block (or its existence in flash) is first verified. If valid, the data is read from flash into the buffer. Otherwise, the buffer remains unmodified, and a failure notification is issued via Fee_JobErrorNotification.



2.8.4 FEE Write

When performing a write operation, the FEE module first checks the validity of the block. If valid, it verifies whether the chunk group associated with the block has sufficient address space. If space is available, the data is written directly. Otherwise, a chunk switch is triggered, and the write operation proceeds after the chunk switch completes.



2.8.5 FEE Cancel

Since all operations of the FLS module on flash are asynchronous, the FEE module cannot determine whether subsequent physical flash addresses remain writable after a cancel operation. Therefore, to ensure software robustness, the Fee_cancel function performs no action.

2.9 Driver usage and configuration tips

Task Scheduling

Make sure that no FEE/FLS functions are interrupted by another FEE/FLS function except Fee_Cancel.

FEE Virtual Page Size

It must be an integral multiple of 8 bytes, for reducing flash space waste, recommended to configure it to 8.

Immediate Data Usage

Immediate data are used for fast write operations, because no chunk switch operation can occur during write (when used properly). Typical use case is storage of crash; related data.

A part of each chunk is reserved for this kind of data. The size of this area is computed from the configuration to be able to hold **one instance** of all immediate data blocks. It is not located in any predefined static area.

Immediate data blocks are usually stored exactly the same way as standard blocks. No pre-allocation of block private data area is performed in the Fee_EraseImmediateBlock function. Only if the given immediate block has already been stored in the reserved area, chunk switch is performed.

Block Always Available

The "Block Always Available" feature has two usage scenarios:

When this option is **false**: If a reset, power loss, or similar event occurs between writing the first part of the header (including the checksum) and writing the valid flag, neither newly written data nor previously written data will be accessible.

When this option is **true**: Even if the aforementioned exception occurs during the write process, the data can still be recovered.

Foreign Blocks feature

The "FEE foreign blocks support" feature can be enabled or disabled by the parameter FeeForeignBlocksSupport.

The " FEE foreign blocks support " feature was developed to support the following customer use case: the customer has two different projects, BOOTLOADER and APPLICATION, which are separately compiled, built into different executable files

which are run on the same ECU in different scenarios. This means the two executables share the same data flash emulated for calibration data. For the two projects the customer uses two different FEE block configurations: for the BOOTLOADER project some blocks, for APPLICATION project other blocks, maybe some blocks are common.

In the previous FEE implementation, it was not possible to run over same data flash with different FEE configurations because FEE used to keep at chunk switch only blocks which were present in the current configuration. This behavior had the disadvantage that each time a new block was added in the APPLICATION project, the customer was required to update also the FEE configuration from the BOOTLOADER project, even if BOOTLOADER blocks were not added or deleted.

The following parameters are used in FeeForeignBlocksSupport mode:

- FeeBlockAssignment Defines in which project this block is used.

  1. BOOTLOADER - Block is used ONLY by the BOOTLOADER project

  2. APPLICATION - Block is used ONLY by the APPLICATION project

  3. Shared - Block is used for BOTH APPLICATION and BOOTLOADER projects

- FeeConfigAssignment Defines for which project the current Fee configuration is used.

  1. BOOTLOADER - Configuration is used only by the BOOTLOADER project

  2. APPLICATION - Configuration is used only by the APPLICATION project

- FeeMaximumNumberBlocks This must be configured to total number of BOOTLOADER, APPLICATION blocks and also some buffer for blocks added in the future project versions. This parameter will be used to statically allocate space for the foreign block information, so it should be configured to accommodate the total number of blocks running on the ECU in all project versions. Example of configuration: If APPLICATION uses 2 blocks, boot loader 1 block and another 1 is shared between APPLICATION and BOOTLOADER then this parameter must be configured to 2(only appl) + 1(only boot loader) + 1(shared) + X, where X is the maximum number of blocks that it is estimated to be added in future project versions.

With FEE in the mode " FeeForeignBlocksSupport " it is possible to have different block configuration for the BOOTLOADER and APPLICATION projects. This is possible by swapping also the foreign blocks. A block is foreign if it has FeeBlockAssignment=BOOTLOADER if the FeeConfigAssignment is APPLICATION or if it has FeeBlockAssignment=APPLICATION if the FeeConfigAssignment is BOOTLOADER.

The mode " FeeForeignBlocksEnabled " is useful in the development phase, but it is not recommended to be used in production phase, as it has the following disadvantages:

- It increases the blank swap overall timing
- It increases the RAM consumption because it needs to allocate management data for the foreign blocks as well.

Nevertheless, it can be used in the production phase if the above points are not critical for the project.

Restrictions for configuring APPLICATION and BOOTLOADER projects:

- If the configurator changes a block which affects the configuration of the other project, then he must apply this change to both configurations. This means:
  - if a SHARED block is changed to APPLICATION only in the APPLICATION configuration or BOOTLOADER only in the BOOTLOADER configuration, then the change must be applied in the other configuration also (delete block from the other configuration)
  - if a non-SHARED block becomes SHARED, then the change must be applied to both configurations The configurator must ensure that block numbers used for BOOTLOADER-only blocks and APPLICATION-only blocks are different.
- If a block is used in both BOOTLOADER and APPLICATION (shared), the block must have the same attributes (block number, size, immediate/non-immediate) in both configurations.

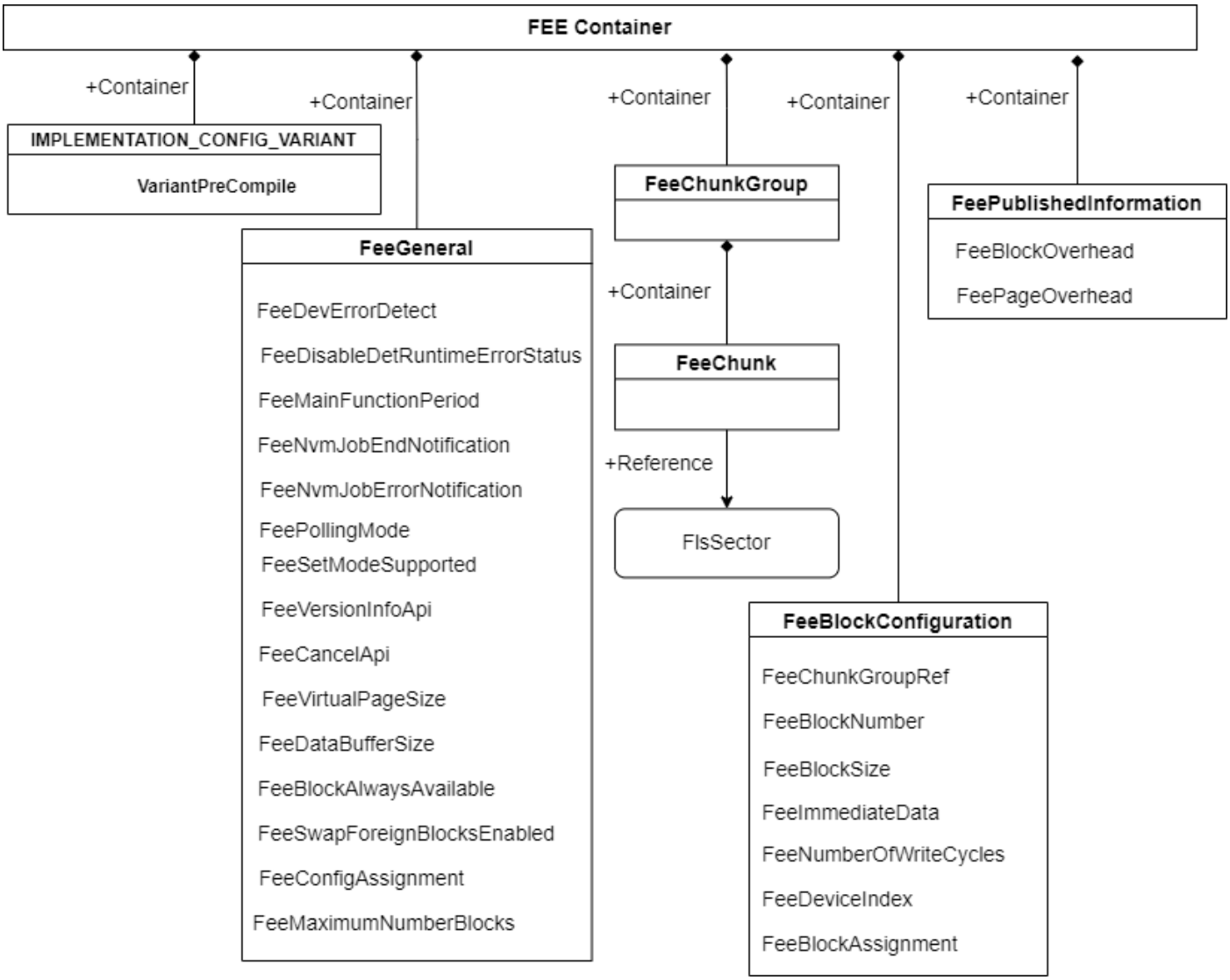
- A block is defined by the following attributes: number, size, immediate/not immediate characteristic. If a part of this information changes in the configuration, then the block will not be copied during the cluster swap.
- All configurations for BOOTLOADER and APPLICATION projects must be the same, except configuration of not-shared blocks which must be different.
- All immediate blocks must be defined as shared between APPLICATION and BOOTLOADER. The reason is that the FEE must have the same reserved area for both configurations.
- The configurator must ensure that the total APPLICATION and BOOTLOADER blocks size with FEE management information included can be accommodated by the data area size.

If FeeForeignBlocksSupport is ON, the block assignment information must be kept in the block header, so the data flash layout will not be compatible with layout with previous FEE versions. This means that the first time when this mode is switched to ON, projects must start with a clean erased data flash.

Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The following chapters summarize all configuration parameters. The detailed meaning of the parameters is shown as below:

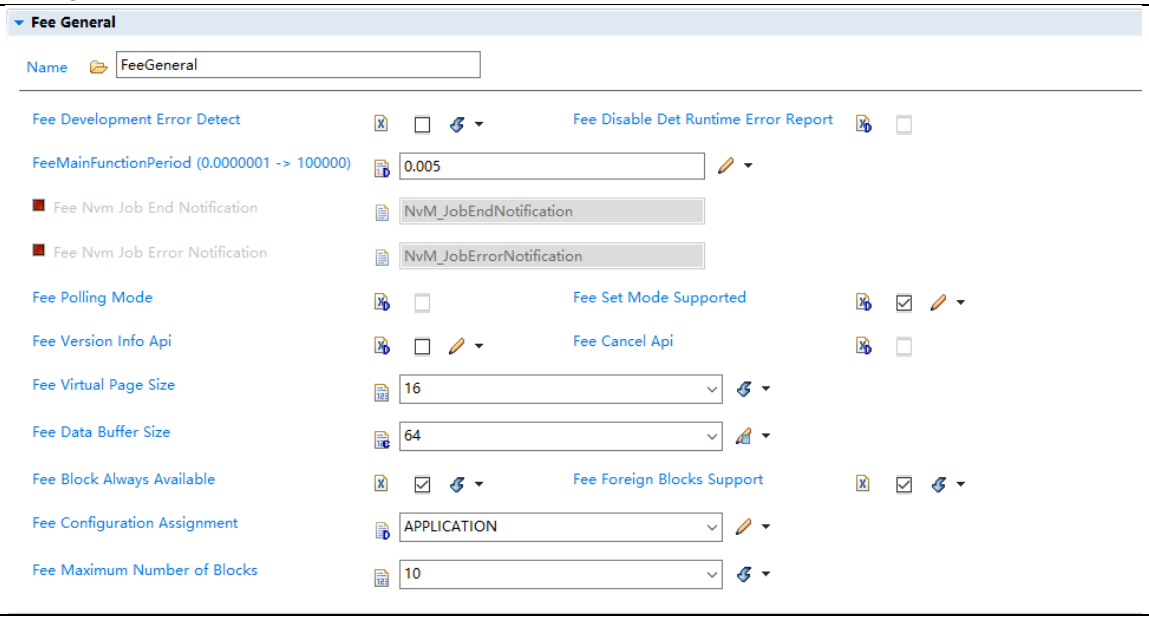


3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot	Config Variant VariantPreCompile	
Properties	**Property**	**Value**
	Type	Variable: enumeration
	Label	Config Variant
	Default	VariantPreCompile
	Range	VariantPreCompile
Returns	N/A	

3.2.2 FeeGeneral

Container	FeeGeneral
Description	Configuration of the Fee (Flash EEPROM Emulation) module.
Screenshot	
Properties	N/A

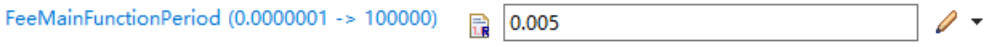
3.2.2.1 FeeDevErrorDetect

Variable	FeeDevErrorDetect	
Description	Switches the development error detection and notification on or off. true: detection and notification is enabled. false: detection and notification is disabled.	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	False

3.2.2.2 FeeDisableDetRuntimeErrorStatus

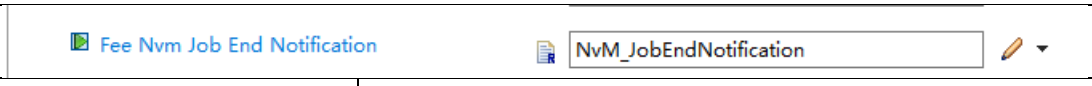
Variable	FeeDisableDetRuntimeErrorStatus	
Description	Enable/Disable Det runtime error reporting.	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	False

3.2.2.3 FeeMainFunctionPeriod

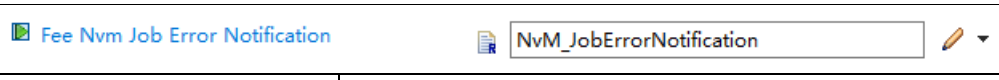
Variable	FeeMainFunctionPeriod	
Description	The period between successive calls to the main function in seconds.	
Screenshot		
Properties	**Property**	**Value**
	Type	FLOAT

	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	5.0E-3
	Range	<=100000, >=1.0E-7

3.2.2.4 FeeNvmJobEndNotification

Container	FeeNvmJobEndNotification	
Description	Mapped to the job end notification routine provided by the upper layer module (NvM_JobEndNotification).	
Screenshot		
Properties	**Property**	**Value**
	Type	FUNCTION-NAME
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	NvM_JobEndNotification

3.2.2.5 FeeNvmJobErrorNotification

Container	FeeNvmJobErrorNotification	
Description	Mapped to the job error notification routine provided by the upper layer module (NvM_JobErrorNotification).	
Screenshot		
Properties	**Property**	**Value**
	Type	FUNCTION-NAME
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	NvM_JobErrorNotification

3.2.2.6 FeePollingMode

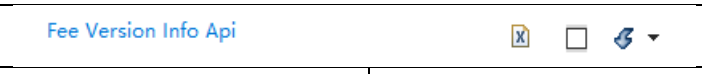
Variable	FeePollingModel	
Description	Pre-processor switch to enable and disable the polling mode for this module. true: Polling mode enabled, callback functions (provided to FLS module) disabled. false: Polling mode disabled, callback functions (provided to FLS module) enabled. Note: This paramater is not utilized in this BSW module implementation	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	False

3.2.2.7 FeeSetModeSupported

Variable	FeeSetModeSupported
Description	Compiler switch to enable/disable the 'SetMode' functionality of the FEE module. true: SetMode functionality supported / code present, false: SetMode functionality not supported / code not present. Note: This configuration setting has to be consistent with that of all underlying flash device drivers (configuration parameter FlsSetModeApi).

Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	False

3.2.2.8 FeeVersionInfoApi

Variable	FeeVersionInfoApi	
Description	Pre-processor switch to enable / disable the API to read out the module' s version information. true: Version info API enabled. false: Version info API disabled.	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	False

3.2.2.9 FeeCancelApi

Variable	FeeCancelApi	
Description	Cancel FEE module current job. Note: For FEE software robustness highly recommended to close this configuration.	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	False

3.2.2.10 FeeVirtualPageSize

Variable	FeeVirtualPageSize	
Description	The size in bytes to which logical blocks shall be aligned	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	16
	Range	<=65535, >=8

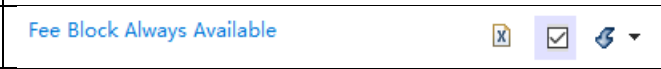
3.2.2.11 FeeDataBufferSize

Variable	FeeDataBufferSize
Description	The data buffer is used to buffer data when Fee is copying data from one cluster to another and when Fee is reading the block header information on startup. Size of the data buffer affects number of Fls_MainFunction cycles. Bigger data buffer improves performance of the Fee cluster management operations and speeds up the startup phase as Fee can read more data in one cycle of Fls_MainFunction

	Note: FeeDataBufferSize must be equal or greater than 4*VIRTUAL_PAGE_SIZE.
Screenshot	

Properties	**Property**	**Value**
	Type	INTEGER
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	32
	Range	<=65535, >=32

3.2.2.12 FeeBlockAlwaysAvailable

Variable	FeeBlockAlwaysAvailable
Description	According to the AUTOSAR requirements, when thewrite operation starts, corresponding block is marked as inconsistent.It is marked as consistent after successful write.It means that when write operation is interrupted (cancel, reset)application cannot access the block data anymore. Enabling this parameter allows to set the behavior against the AUTOSARrequirements. In this case, the last valid information is always provided.Valid information means: •FEE_BLOCK_INVALID if the block was invalidated or the block isnot present in the chunk at all. •FEE_BLOCK_VALID in case the previous instance of the block exists andwas not invalidated.
Screenshot	

Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	True

3.2.2.13 FeeSwapForeignBlocksEnabled

Variable	FeeSwapForeignBlocksEnabled
Description	The recommendation is not to use this mode Enabled for the production phase as it increases the chunk switch overall timing. This mode may be useful for development phase if Fee is used in 2 projects running on the same ECU and the integrator wants to be able to change configuration for the application project without changing it for the second project. If disabled, the Fee driver will swap only blocks existing in the current configuration. If enabled, the Fee driver will swap foreign blocks. This is a limited support when using 2 subprojects with different Fee configurations running on the same ECU (example: boot loader and applications projects). For example, if this parameter is enabled and Fee is running in application 1 mode, it will swap blocks assigned to application 2, even if those blocks are not present in application 1 configuration. This allows to update the Fee configuration for application 2 without reconfiguring application 1. Nevertheless, this mode will result in a longer time for blank swap.
Screenshot	

Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	False

3.2.2.14 FeeConfigAssignment

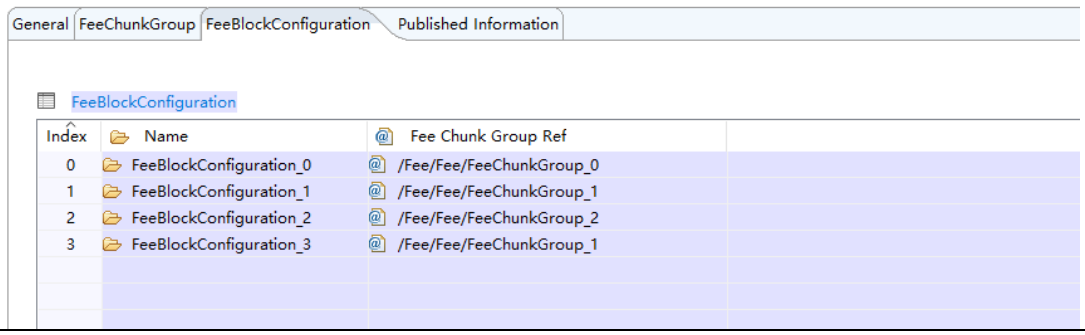
Variable	FeeConfigAssignment

Description	Choose for which project this Fee configuration is used. BootLoader - Configuration is used only by the boot loader project Application - Configuration is used only by the application project	
Screenshot		
Properties	**Property**	**Value**
	Type	ENUMERATION
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	APPLICATION
	RANGE	APPLICATION, BOOTLOADER

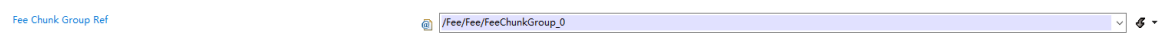
3.2.2.15 FeeMaximumNumberBlocks

Variable	FeeMaximumNumberBlocks	
Description	This must be configured to total number of boot loader, application blocks and also some buffer for blocks added in the future. This parameter will be used to statically allocate space for the block runtime information, so it should be configured to accommodate the total number of blocks running on the ECU in all project versions. The parameter is used to support Fee Swap Foreign Blocks Feature.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER
	Origin	FLAGCHIP
	SymbolicNameValue	True
	Default	1
	RANGE	<=65534, >=1

3.2.3 FeeBlockConfiguration

Variable	FeeBlockConfiguration
Description	Configuration of block specific parameters for the Flash EEPROM Emulation module
Screenshot	
Properties	N/A

3.2.3.1 FeeChunkGroupRef

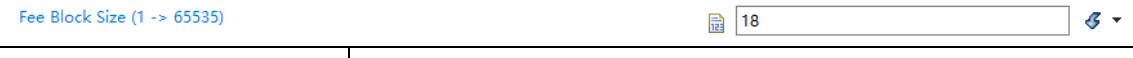
Variable	FeeChunkGroupRef	
Description	Reference to the Fee chunk group which the Fee block belongs to. In other words, FeeChunkGroupRef assigns the Fee block to particular Fee chunk group.	
Screenshot		
Properties	**Property**	**Value**
	Type	Reference
	Origin	FLAGCHIP
	Symbolic Name Value	True

	Range	N/A

3.2.3.2 FeeBlockNumber

Variable	FeeBlockNumber	
Description	Block identifier (handle). 0x0000 and 0xFFFF shall not be used for block numbers (see FEE006). Range: min = 2^NVM_DATASET_SELECTION_BITS max = 0xFFFF -2^NVM_DATASET_SELECTION_BITS Note: Depending on the number of bits set aside for dataset selection several other block numbers shall also be left out to ease implementation.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	True
	Range	<65535, >=1

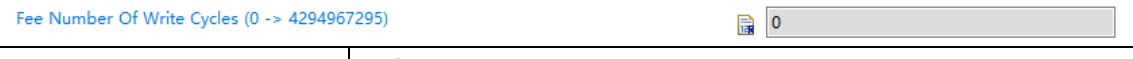
3.2.3.3 FeeBlockSize

Variable	FeeBlockSize	
Description	Size of a logical block in bytes.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	1
	Range	<=65535, >=1

3.2.3.4 FeeImmediateData

Variable	FeeImmediateData	
Description	Marker for high priority data. true: Block contains immediate data. false: Block does not contain immediate data.	
Screenshot		
Properties	**Property**	**Value**
	Type	BOOLEAN
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	False
	Default	False

3.2.3.5 FeeNumberOfWriteCycles

Container	FeeNumberOfWriteCycles	
Description	Number of write cycles required for this block. Note: This parameter is unused in the given implementation.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER

	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	0
	Range	<=4294967295, >=32

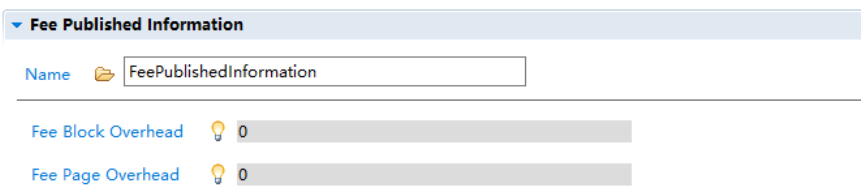
3.2.3.6 FeeDeviceIndex

Variable	FeeDeviceIndex	
Description	Device index (handle). Range: 0 .. 254 (0xFF reserved for broadcast call to GetStatus function).	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER
	Origin	SYMBOLIC-NAME-REFERENCE
	SymbolicNameValue	False

3.2.3.7 FeeBlockAssignment

Variable	FeeBlockAssignment	
Description	Choose to which project this block is assigned. BootLoader - Block is used only by the boot loader project Application - Block is used only by the application project Shared - Block is used for the Application and Boot Loader projects	
Screenshot		
Properties	**Property**	**Value**
	Type	ENUMERATION
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	APPLICATION
	RANGE	APPLICATION, BOOTLOADER, SHARED

3.2.4 FeePublishedInformation

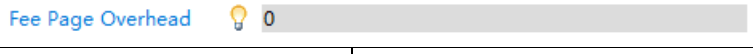
Variable	FeePublishedInformation
Description	Additional published parameters not covered by CommonPublishedInformation container. Note that these parameters do not have any configuration class setting, since they are published information.
Screenshot	
Properties	N/A

3.2.4.1 FeeBlockOverhead

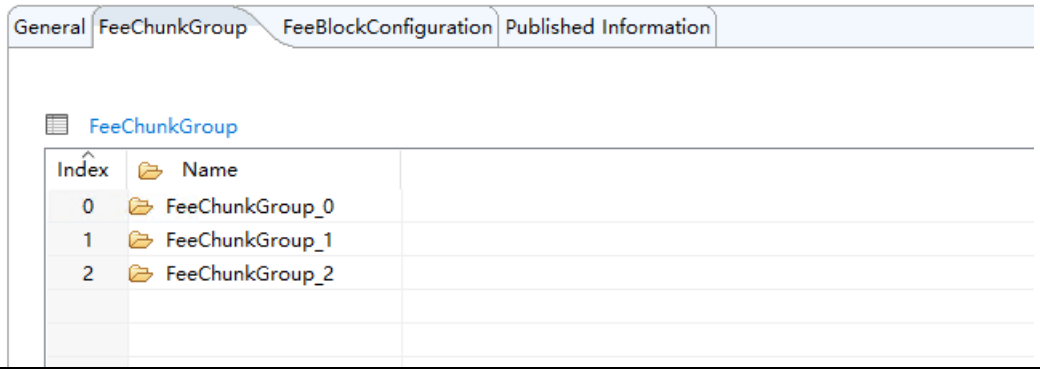
Container	FeeBlockOverhead	
Description	Management overhead per logical block in bytes. Note: The logical block management overhead depends on FeeVirtualPageSize and can be calculated using the following formula: ceiling(12 / FEE_VIRTUAL_PAGE_SIZE + 2) * FEE_VIRTUAL_PAGE_SIZE	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER_LABEL

	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	0
	Range	<=65535, >=0

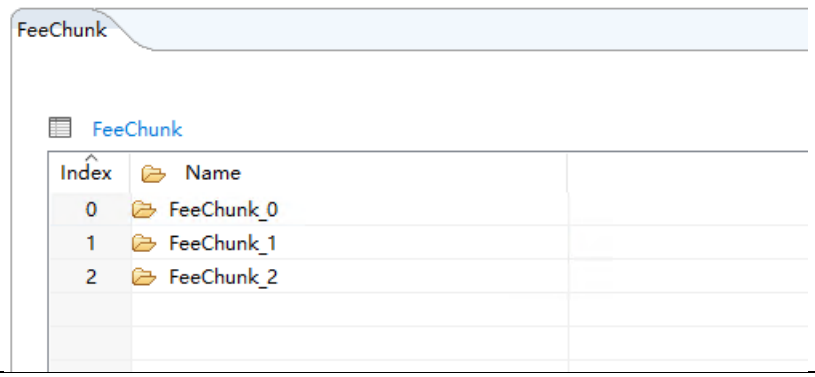
3.2.4.2 FeePageOverhead

Variable	FeePageOverhead	
Description	Management overhead per page in bytes. Note: The page management overhead is 0 bytes	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	AUTOSAR_ECUC
	Symbolic Name Value	False
	Default	0
	Range	<=65535, >=0

3.2.5 FeeChunkGroup

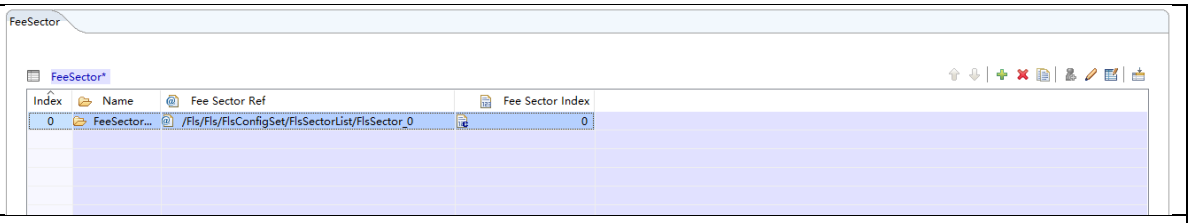
Container	FeeChunkGroup
Description	Configuration of chunk group specific parameters for the Flash EEPROM Emulation module.
Screenshot	
Properties	N/A

3.2.6 FeeChunk

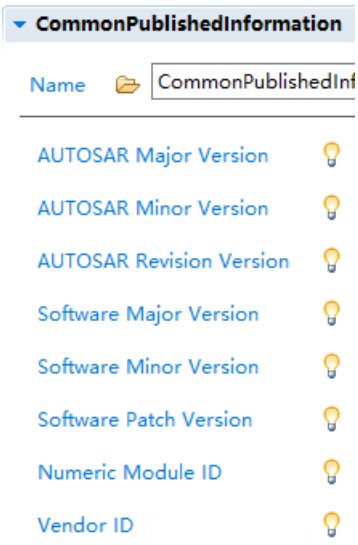
Container	FeeChunk
Description	Configuration of chunk specific parameters for the Flash EEPROM Emulation module.
Screenshot	
Properties	N/A

3.2.6.1 FeeSector

Variable	FeeSector
Description	Select Fls physical sector for logic sector of header area

Screenshot	
Properties	N/A

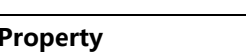
3.2.7 CommonPublishedInformation

Container	CommonPublishedInformation
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.
Screenshot	
Properties	N/A

3.2.7.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	4

3.2.7.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	6

3.2.7.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion

Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot	AUTOSAR Revision Version	
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	0

3.2.7.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Major Version	
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	1

3.2.7.5 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Minor Version	
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	2

3.2.7.6 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot	Software Patch Version	
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	0

3.2.7.7 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot	Numeric Module ID	
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP

	Symbolic Name Value	False
	Default	21

3.2.7.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	**Property**	**Value**
	Type	INTEGER_LABEL
	Origin	FLAGCHIP
	Symbolic Name Value	False
	Default	174

Chapter 4 Configuration Guides

4.1 Configuration Steps

The FEE module depends on the underlying FLS module and its configuration, so FLS sectors must be configured before FEE is used.

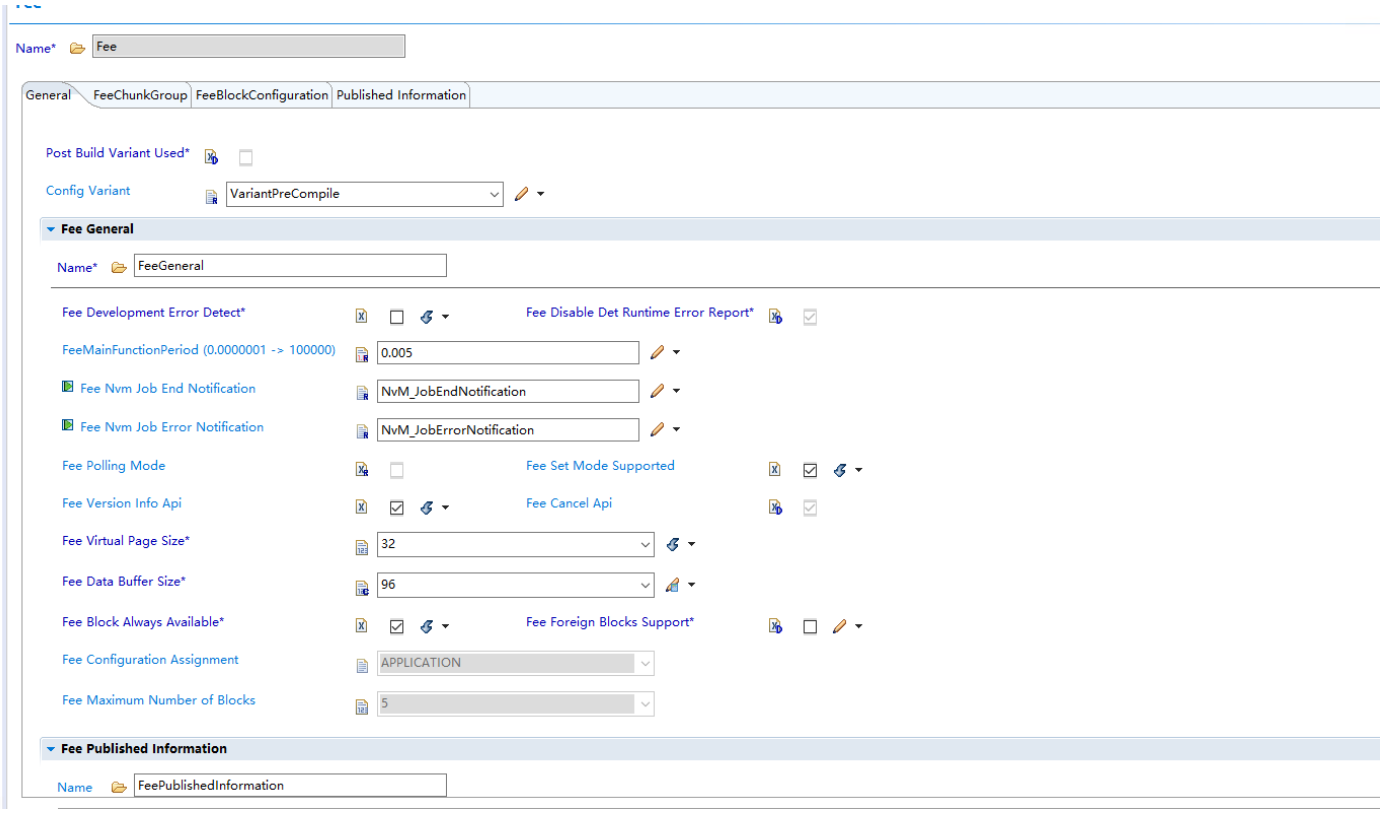
The FEE module can be configured as 5 steps after FLS is ready:

1) FeeGeneral containers configure;
2) FeePublishedInformation containers configure;
3) FeeChunkGroup containers configure;
4) FeeChunk containers configure;
5) FeeBlockConfiguration containers configure.

4.2 Configuration Contents

4.2.1 FeeGeneral

FeeGeneral containers configure according to the actual use, typically as follows:



Tips:

- FeeDevErrorDetect: Open for error checking;
- FeeDisableDetRuntimeErrorStatus: Open for error checking;
- FeeMainFunctionPeriod: Set time according to actual usage (unit: Second);
- FeeNvmJobEndNotification: Open for Nvm use;
- FeeNvmJobErrorNotification: Open for Nvm use;
- FeePollingModel: Not used;

- FeeSetModeSupported: Open for setting flash mode, this configuration setting has to be consistent with FlsSetModeApi;
- FeeVersionInfoApi: Open for getting software version;
- FeeCancelApi: Default unused;
- FeeVirtualPageSize: For reducing flash space waste, recommended to configure it to 16;
- FeeDataBufferSize: Minimum of 4 * FEE_PAGE_OVERHEAD.

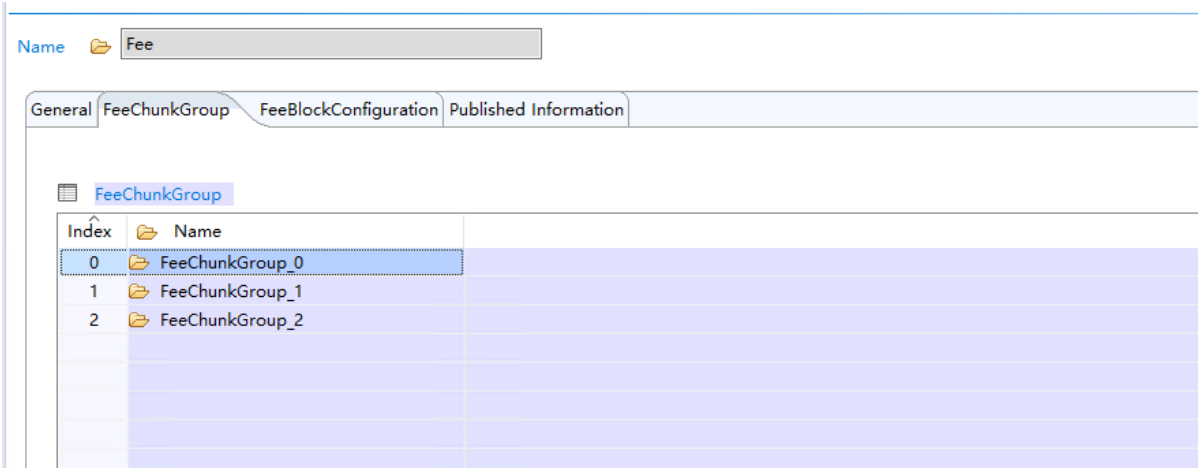
4.2.2 FeePublishedInformation

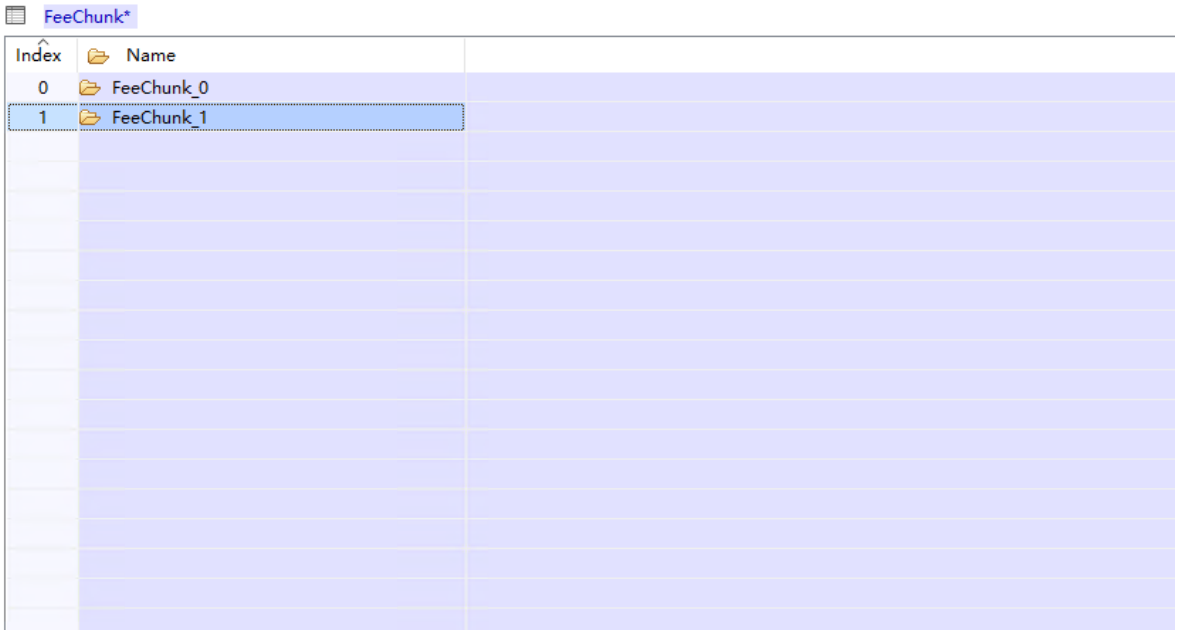
FeeBlockOverhead and FeePageOverhead are calculated automatically, the calculation formula are:

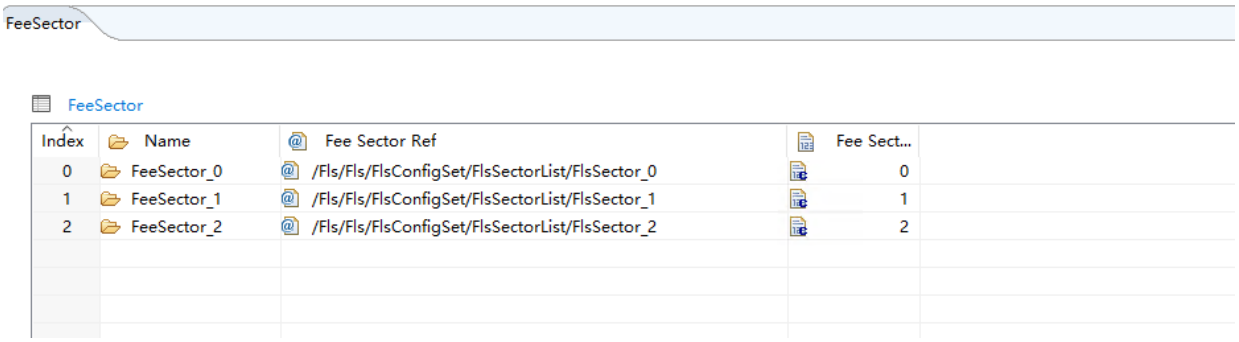
- FeeBlockOverhead = 4 * FeeVirtualPageSize
- FeePageOverhead = 4 * FeeVirtualPageSize

4.2.3 FeeChunkGroup

FeeChunkGroup can be configured to several groups, and each group contains several chunks.







Constraints:

1) The number of Chunk groups is theoretically infinite, as long as the flash space is large enough.
2) The number of Chunk shall more than 2 for chunk switch.
3) Crossing use of Fls sector is not allowed.

4.2.4 FeeBlockConfiguration

Configuration of block-specific parameters for the FEE module. Details depend on the actual scenario.